

Master Thesis

From Skills to Survival: Hierarchical Composition of Neuroevolved Skills for Long-Horizon Reinforcement Learning

by

Sjoerd Holtus
(2864921)

Supervisor: Kevin Godin-Dubois
Second Assessor: Kevin Sebastian Luck

August 2026

Submitted in partial fulfilment of the requirements for
the VU degree of Master of Science in Artificial Intelligence

Abstract. Hierarchical reinforcement learning can ease exploration and credit assignment in long-horizon tasks by composing reusable lower-level behaviours. This thesis introduces Meta-PPO, which combines a proximal policy optimization (PPO) meta-controller with six independently neuroevolved, frozen skills in Craftax Classic, a procedurally generated survival-and-crafting benchmark. Across five runs, Meta-PPO achieves a higher mean normalized environment return than Flat PPO at downstream budgets up to two million steps ($36.1 \pm 3.8\%$ versus $33.0 \pm 3.3\%$), before Flat PPO overtakes it. After twenty million steps, Meta-PPO retains 96.3% of Flat PPO's mean training return while using 83.0% fewer parameters. The recurrent neural network (RNN) variant Meta-PPO-RNN likewise exceeds Flat PPO-RNN during training ($58.7 \pm 2.5\%$ versus $39.0 \pm 7.7\%$) and numerically matches feed-forward Flat PPO in held-out greedy evaluation (both 58.3%); its complete hierarchy uses 86.0% fewer parameters than Flat PPO and 89.7% fewer than Flat PPO-RNN. Skill acquisition is substantially more expensive than downstream training, but the resulting frozen library remains modular: the same fixed skills can be used across controllers and objectives and transfer with varying success to the benchmark's harder Extended variant. Further analyses favour one-step invocation over fixed temporal commitment. Overall, the results support compact, modular control through reusable skills whose value depends on the training budget and task setting.

Keywords: Hierarchical reinforcement learning · Neuroevolution · PPO · Skill composition · Craftax

1 Introduction

Reinforcement-learning agents struggle when reward requires long sequences of exploration, resource acquisition, survival, and interaction. A flat policy must learn both which intermediate behaviours are useful and how to execute them. Hierarchical reinforcement learning (HRL) instead lets a high-level controller compose lower-level behaviours, or skills, potentially easing exploration and credit assignment.

Craftax Classic [13] is a procedurally generated survival and crafting benchmark with 22 interdependent achievements. Long prerequisite chains, local observations, and changing maps require both high-level progression and reliable local interaction. This thesis asks whether those local interaction patterns can be acquired before downstream Proximal Policy Optimization (PPO) [19]. Six compact policies are trained independently with OpenAI's Evolution Strategies (OpenES) [18], task-specific fitness functions, feature vectors, and bootstrapped environments. Their parameters are frozen and composed by a PPO meta-controller. Each selected skill produces one primitive action before control returns to the controller, providing behavioural reuse without the temporal commitment of a conventional option [21]. The resulting agent is denoted Meta-PPO and compared with the original Craftax feed-forward PPO baseline, denoted Flat PPO.

The central question addressed by this thesis is therefore:

How does a PPO meta-controller that composes independently trained, frozen neuroevolved skills compare with Flat PPO across downstream training budgets in Craftax Classic?

Across five runs, Meta-PPO has a higher observed mean return through two million downstream steps; Flat PPO then overtakes it, but Meta-PPO retains 96.3% of its final mean training return with 83.0% fewer parameters. Meta-PPO-RNN substantially outperforms Flat PPO-RNN during training and numerically matches feed-forward Flat PPO in held-out greedy evaluation while using 86.0% fewer parameters. Skill-level studies further test fixed execution horizons and transfer to the more complex Craftax Extended variant. Together, the experiments assess not only downstream performance, but also parameter efficiency, behavioural reuse, and the cost of the skill interface.

2 Related Work

2.1 Deep Reinforcement Learning and Policy Architectures

Deep reinforcement learning uses neural networks to approximate policies, value functions, or both. The deep Q-network (DQN) demonstrated that one architecture could learn action-value functions from pixels across multiple Atari games [15]; actor–critic and policy-gradient algorithms subsequently learned explicit policies. PPO [19] is widely used because its clipped objective supports several optimisation epochs while limiting destabilising policy changes.

For structured observations, classical feed-forward networks map the current input directly to action logits and values but retain no history. Recurrent neural networks (RNNs) instead maintain a hidden state across environment steps. Hausknecht and Stone [8] found that adding recurrence to DQN did not systematically help under full observability, but degraded more gracefully when observations were obscured. This suggests that RNNs can be effective under partially observable conditions.

2.2 Hierarchical Control and Reusable Skills

The options framework [21] formalises HRL through options, each defined by an initiation set, internal policy, and termination condition. After a high-level policy selects an option, its internal policy controls primitive actions until termination. Options often introduce *temporal abstraction* by allowing a selected behaviour to execute for multiple environment steps before control returns to the higher-level policy, although they may also terminate after a single primitive action. Such a hierarchy enables behavioural reuse while retaining the environment's native control frequency.

Many HRL methods learn the hierarchy jointly on the downstream task. Option-Critic [1] learns intra-option policies and termination rules; h-DQN [11]

combines a goal-selecting meta-controller with an intrinsically rewarded lower-level controller; FeUdal Networks [22] divide control between a latent-space Manager and Worker. Unsupervised methods such as DIAYN [5] instead acquire diverse behaviours before a downstream task is specified.

Frozen lower-level behaviours can instead be acquired separately. Matthews et al. distil expert policies and evaluate a controller that selects among them [14]. Chalumeau et al. compose neuroevolved locomotion skills with PPO in a hurdle-crossing variant of *HalfCheetah*; each selected skill executes for ten steps [2]. These freeze-then-compose systems separate learning a behaviour from learning when to invoke it.

The present work combines frozen skills with one-step execution. This distinction matters because Nachum et al. [17] show that hierarchical gains can arise from exploration and favourable training distributions rather than temporal abstraction itself.

2.3 Neuroevolution and Policy Architectures

Neuroevolution optimises neural-network parameters through fitness rather than backpropagating through trajectories. Salimans et al. [18] showed that evolution strategies can train high-dimensional policies competitively and parallelise efficiently; Such et al. [20] reported similar results with genetic algorithms. Because only a scalar objective is required, different skills can use different sparse, delayed, or non-differentiable objectives and be trained before a controller exists.

Chalumeau et al. [2] provide the closest precedent for this combination of neuroevolved skills and a PPO controller. Their continuous locomotion setting, however, has low-dimensional observations and temporally extended execution. Craftax tests whether compact neuroevolved skills remain composable in a discrete, partially observable world with procedural variation, long prerequisite chains and open-ended survival.

2.4 Crafter, Craftax, and Positioning

Crafter introduced a lightweight, procedurally generated survival benchmark spanning exploration, resource gathering, crafting, combat, and survival [6]. Craftax reimplemented it in JAX for highly parallel training and added a substantially harder Extended variant [13]. The Craftax benchmark provides both feed-forward and recurrent PPO baselines.

Work on these benchmarks includes Achievement Distillation, which adds a contrastive objective for predicting the next achievement [16]; DreamerV3, which learns a latent world model and trains from imagined trajectories [7]; and transformer-based world modelling that combines real and imagined experience [4]. These methods improve representations or planning within end-to-end agents rather than composing separately acquired behaviours.

The present work instead studies whether compact, independently neuroevolved skills can support one-step composition by PPO in a procedurally generated, partially observable task with long prerequisite chains. It combines the

freeze-then-compose setting of Matthews et al. [14] with the neuroevolved skill library of Chalumeau et al. [2], but removes fixed temporal commitment and evaluates both feed-forward and recurrent controllers.

Three distinctions define the resulting research gap. First, most end-to-end Craftax agents learn one representation and policy for the entire achievement graph; they do not test whether local behaviours can be acquired under separate objectives and subsequently reused. Second, prior freeze-then-compose work generally assumes expert demonstrations or temporally extended skills, whereas this thesis acquires its library directly through fitness and returns control after every primitive action. Third, parameter efficiency is rarely central to these comparisons. Here, the complete hierarchy includes both controller and frozen skill parameters, which allows behavioural reuse to be assessed alongside model size rather than performance alone.

Furthermore, this thesis studies engineered skill composition rather than automatic option discovery. It asks whether a library of separately trained, frozen skills can provide useful reusable structure to compact PPO controllers, and how that value changes across training budgets and task settings.

3 Background

3.1 Hierarchical Reinforcement Learning

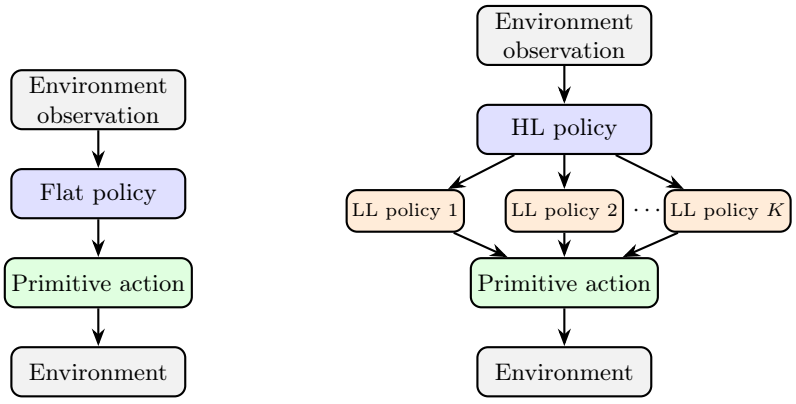
In flat reinforcement learning, a single policy maps observations directly to primitive environment actions and must learn both how to execute behaviours and how to sequence them. HRL separates these responsibilities: a high-level policy selects among lower-level policies, or skills, which in turn determine primitive actions. Consequently, the high-level action space can include meaningful behaviours rather than only primitive actions. Figure 1 illustrates this distinction.

This hierarchy can introduce two distinct forms of abstraction. *Behavioural abstraction* exposes a learned lower-level policy as a selectable action to the higher-level controller. *Temporal abstraction* allows that behaviour to persist across environment steps. Its *execution horizon* h is the number of steps before control returns; the primary design uses $h = 1$, so the controller switches after every primitive action.

Beyond abstraction, *modularity* provides another motivation for HRL. If a lower-level policy depends only on task-relevant features, it can potentially be reused across targets or environments. Freezing such a policy prevents downstream learning from destroying the behaviour, but also prevents correcting it when its features or training objective omit relevant context.

3.2 Proximal Policy Optimization

PPO [19] is an on-policy actor–critic algorithm. The actor defines a parameterised distribution over actions, while the critic estimates the expected future return from a state. The critic supplies a baseline for an advantage estimate



(a) Flat reinforcement learning (b) Hierarchical reinforcement learning

Fig. 1: Conceptual comparison of flat and hierarchical reinforcement learning. A flat policy selects primitive actions directly, whereas a hierarchical policy uses a higher-level (HL) policy to select a lower-level (LL) policy, which then determines the primitive action.

$\hat{A}_t$, which indicates whether the sampled action performed better or worse than expected.

Large policy-gradient updates may change action probabilities abruptly and destabilise learning. PPO limits this by comparing the current policy with the policy that collected the rollout. With $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$, it optimises

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right], \tag{1}$$

where ϵ is the clipping threshold. Clipping removes the incentive to move the probability ratio further once it leaves the allowed interval in a direction that would increase the surrogate objective. The full optimisation also includes a value-function loss and an entropy bonus that discourages premature collapse of the action distribution. Multiple epochs can be taken over each on-policy batch, subject to this conservative update.

3.3 Recurrent Policies

Feed-forward policies condition only on the current observation. In a partially observable environment, identical local observations can require different actions depending on earlier events. A recurrent policy therefore updates a hidden state,

$$h_t = f(o_t, h_{t-1}), \tag{2}$$

and conditions its actor and critic on h_t. The hidden state acts as a learned summary of observation history.

Standard recurrent networks can struggle to retain information over long sequences. Long short-term memory (LSTM) networks use gated memory cells to preserve and update information [9]. Gated recurrent units (GRUs) use reset and update gates for the same general purpose [3]. GRUs have a simpler gating structure than LSTMs and are used by the Craftax PPO-RNN baseline evaluated here.

3.4 Evolution Strategies and OpenES

Evolution strategies (ES) treat policy parameters as the search variable and episodic performance as scalar fitness. They evaluate sampled parameter vectors rather than differentiating through trajectories, so the environment and objective need not be differentiable [23]. This is useful for skill objectives that combine discrete events, distances, and terminal outcomes.

OpenES [18] maintains a Gaussian search distribution

$$p_\phi(\theta) = \mathcal{N}(\mu, \sigma^2 I), \tag{3}$$

with search mean μ and exploration scale σ. A population is sampled as $\theta_i = \mu + \sigma\epsilon_i$, with $\epsilon_i \sim \mathcal{N}(0, I)$. Antithetic (mirrored) pairs evaluate both ϵ_i and $-\epsilon_i$, reducing variance in the estimated update. Each parameter vector defines a policy that is rolled out on one or more environments.

Raw fitness is replaced by centred ranks. The update therefore depends on the ordering of candidates rather than the absolute scale of a skill's fitness, reducing sensitivity to outliers and allowing task-specific objectives to have different numeric ranges. OpenES estimates

$$\nabla_\mu \mathbb{E}_{\theta\sim p_\phi}[F(\theta)] \approx \frac{1}{n\sigma} \sum_{i=1}^{n} F_i^{\text{rank}}\, \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, I), \tag{4}$$

where F_i^{rank} is candidate i's rank-normalised fitness and n is population size. Adam [10] applies the estimated gradient to μ, after which a new population is sampled. Candidate evaluations are independent and can be vectorised in JAX. This parallelism and the skills' small networks partly offset the large number of environment transitions used by ES.

3.5 JAX and Craftax Classic

JAX is a Python library for compiled, vectorised numerical computation; its `vmap` transformation batches function evaluations, while `jit` compiles functions for accelerated execution. Craftax Classic [13] builds on this model to reimplement Crafter [6] as a highly parallel survival benchmark for reinforcement learning research.

Each episode generates a new world containing terrain, resources, passive animals, and hostile mobs. The agent gathers materials, crafts tools, fights enemies, and balances health, hunger, thirst, and energy. Depletion of hunger, thirst or

energy damages health, so immediate maintenance competes with longer-term progression. The achievement dependencies in Figure 2 make this progression sequential: collecting iron requires a stone pickaxe, whose construction first requires wood, a table, and stone.

The agent observes a local 7×9 region rather than the full map. One-hot block and entity channels are concatenated with inventory, equipment, status, and orientation, producing a 1345-dimensional symbolic vector. Limited visibility and procedural variation make the task partially observable and require generalisation across layouts. The 17 native actions cover movement, contextual interaction (`DO`), sleep, placement, and crafting (Appendix B); the observation construction appears in Appendix E.

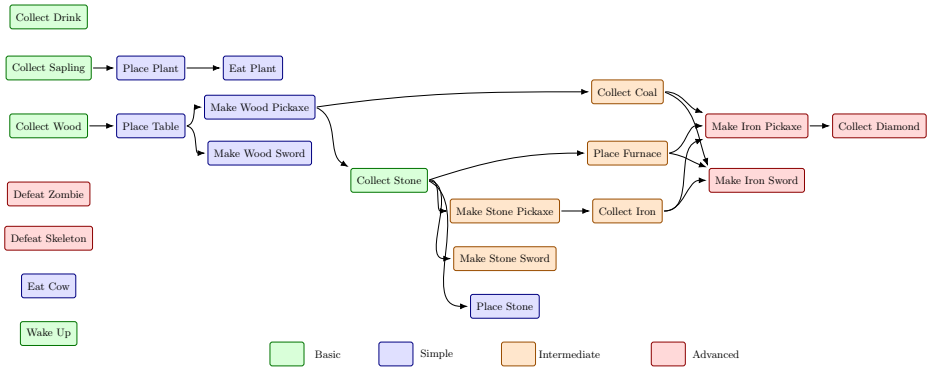


Fig. 2: Craftax achievement dependencies, colour-coded by approximate difficulty.

Each of 22 achievements yields +1 on first completion; health changes add 0.1 per point [13]. Thus

$$r_t = \sum_{i=1}^{22} \mathbb{I}\left[a_{i,t+1} = 1 \wedge a_{i,t} = 0\right] + 0.1\left(h_{t+1} - h_t\right), \tag{5}$$

where $a_{i,t} \in \{0, 1\}$ indicates whether achievement i has been completed and h_t is health. An achievement can reward the agent only once per episode, whereas damage and recovery contribute throughout. The resulting reward structure therefore combines sparse rewards for prerequisite-dependent progression with a denser reward signal for survival.

3.6 Craftax Extended

Craftax Extended adds nine procedurally generated floors, 67 weighted achievements, and new systems for combat, equipment, crafting, magic, and character progression [13]. It also increases the local observation field of view from 7×9

to 9×11. Together, these changes lengthen task dependencies and expand the symbolic observation from 1345 to 8268 features. The primary controller benchmark in this thesis is the Classic variant; a supplementary study tests whether the same lower-level weights retain competence in the Extended variant when the feature filters (subsection 4.2) are remapped to the larger observation.

4 Method

The proposed hierarchy combines independently trained, frozen lower-level policies (*skills*) with a PPO high-level policy (the *meta-controller*) to form *meta-PPO*.

4.1 Hierarchical Architecture

The skill library contains *Collect Wood*, *Find Water*, *Hunt Food*, *Combat*, *Collect Stone*, and *Collect Ore*. Each is a small feed-forward policy that selects from

$$\mathcal{A}_{\text{skill}} = \{\text{LEFT}, \text{RIGHT}, \text{UP}, \text{DOWN}, \text{DO}\}. \tag{6}$$

The ore policy is trained with coal as its target and reused for iron and diamond by changing the feature filter. Six networks consequently implement eight skill-backed meta-actions.

The meta-action space also contains 16 direct actions: no-op, movement, sleep, placement, and crafting (Appendix C). At step t, the controller observes the full symbolic vector o_t and samples

$$m_t \sim \pi_\theta(m_t \mid o_t), \qquad m_t \in \{0, \ldots, 23\}. \tag{7}$$

All 24 choices remain available in every state.

For a skill-backed choice, a feature filter extracts a compact, task-specific vector. Based on this feature vector, the selected frozen policy produces one native action from Equation 6. Direct choices map deterministically to their native action. The native `DO` interaction is deliberately unavailable as a direct meta-action, so gathering, mining, drinking, and attacks must pass through a skill. The hierarchy is therefore a necessary part of the control architecture rather than an optional side channel.

Each selection advances the environment by exactly one step. The controller then receives the next observation and may choose any skill-backed or direct action. Figure 3 summarises the data and control flow.

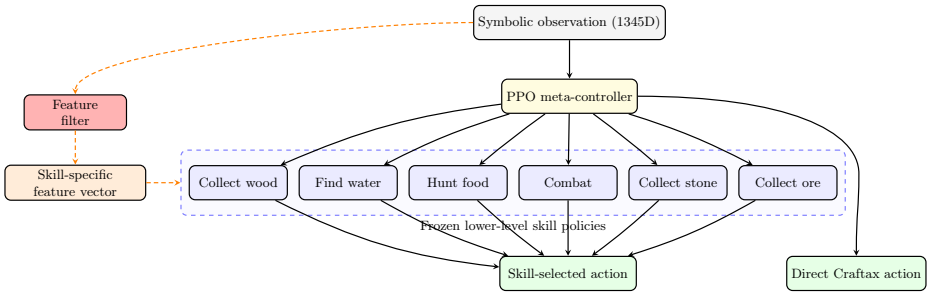


Fig. 3: Hierarchical control architecture. The PPO meta-controller receives the full 1345-dimensional observation and selects a skill-backed or direct meta-action. Skill-backed choices pass a filtered vector to a frozen policy, which outputs a native action; direct choices map directly to native actions.

4.2 Lower-Level Skill Training

Each skill is trained independently with `evosax.Open_ES` [12]. It solves one restricted task rather than complete survival, and has its own fitness, bootstrap, feature vector, policy size, and training schedule. Independent acquisition permits the behaviours to be developed and inspected before any downstream controller is introduced.

Fitness and Bootstrapping Fitness for gathering policies is dominated by increases in target resource. Ore additionally rewards removing stone between the agent and its target, because successful mining may require several preparatory interactions. Hunting and combat provide a large one-time kill bonus plus smaller rewards for successful attacks and progress towards the nearest target; wasted attacks are lightly penalised while a target remains. These dense terms provide direction before OpenES samples a complete approach–face–interact sequence.

At reset, each bootstrapped environment places relevant resources or mobs within controlled distances (Figure 4). Non-combat bootstraps remove unrelated threats so that survival noise does not dominate the local objective. The combat bootstrap instead supplies zombies, skeletons, equipment, and obstacles. Bootstrap modifications are used only for skill acquisition.

Feature Vectors and Curricula Skills do not operate on the full 1345-dimensional observation. Instead, each receives a compact, skill-specific vector containing target direction and distance, local walkability, adjacency, facing, and, for five of the six skills, a task-specific resource or health signal. *Hunt Food* is the exception: its ten features are entirely directional and spatial, because the kill events that drive its fitness are not observable as an inventory or status quantity. Every feature is derived from the agent's local symbolic observation; no global state is exposed. Shared directional and navigation features are reused across skills, while target channels and status features vary by task.



Fig. 4: Bootstrapped *Hunt Food*, *Find Water*, and *Combat* environments. Task-relevant targets are placed locally during skill training only.

The fixed feature interface enables target substitution: the ore extractor can point to the coal, iron, or diamond channel without changing the trained policy weights. It also introduces a deliberate inductive bias, since information outside the vector cannot influence the skill. This improves compactness but can become a downstream limitation when omitted context matters.

The filter is therefore the principal interface between the full environment observation and a skill. It converts spatial channels into relative target and obstacle features, appends task status variables, and presents a fixed-length vector to the small policy. The policy never sees which meta-action invoked it or the controller's state. This separation makes checkpoints portable: retargeting the ore skill changes the selected block channel, while leaving the network and action mapping untouched. Figure 5 shows the pipeline for the 18-dimensional combat skill vector. Appendix F illustrates the complete feature-vector construction from the full observation to the frozen skill policies.

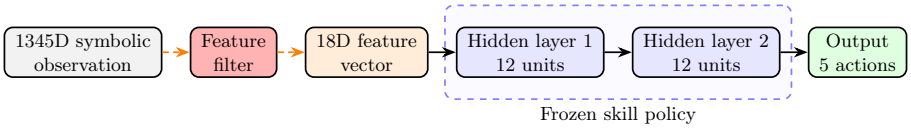


Fig. 5: Combat skill pipeline. The symbolic observation is filtered into an 18-dimensional task-specific vector, which a two-layer network maps to one of five primitive actions.

Training difficulty increases gradually. Rollout counts rise over generations while perturbation scale, policy temperature, and stochastic action selection are annealed. Food and combat use two 100-generation phases: stationary mobs first encourage active seeking, then normally moving mobs train pursuit. The second phase is warm-started from the first. Ore uses a spatial curriculum that increases

the probability of thicker stone shells around coal (Figure 6). These schedules reduce passive or short-sighted local optima.



Fig. 6: Ore curriculum with progressively thicker stone shells. The policy must first learn direct interaction and later learn to clear an approach to the target.

The curricula differ because the probability of observing useful fitness differs by task. A short gathering rollout often contains several attempts at a static target, whereas food and combat require searching for a moving entity, approaching it, repeatedly orienting and interacting, and surviving retaliation. Their longer 90-step horizons and two training phases consequently dominate the transition count. Table 1 reports this cost explicitly rather than treating all six checkpoints as equally expensive.

Table 1: Training budgets for the lower-level skills. Food and combat use two phases and account for 91.3% of the exact 259.2M-step total; displayed per-skill budgets are rounded.

Skill	Phases	Generations	Horizon	Env. steps
Collect Wood	1	100	20	2.1M
Find Water	1	100	20	6.8M
Hunt Food	2	100+100	90	118.3M
Combat	2	100+100	90	118.3M
Collect Stone	1	100	20	6.8M
Collect Ore	1	200	10	6.8M
Total				**259.2M**

The complete fitness functions, bootstrap parameters, feature definitions, network architectures, and OpenES schedules are given in Appendix D. The final search means are greedily refined during the last 20–25% of training to match downstream argmax execution, then frozen before PPO training.

Checkpoint construction is deliberately independent of downstream PPO performance. OpenES maintains a search mean rather than a single sampled individual, and this mean is the policy transferred to the hierarchy. During the final greedy tail, action sampling is replaced by argmax selection while evolution continues, reducing a train–execution mismatch without consulting controller return. The resulting six checkpoints are then fixed once and shared by Meta-PPO, Meta-PPO-RNN, all five downstream seeds, and the transfer experiments. No checkpoint is selected separately for a favourable controller run. All hierarchical runs use the same frozen skill library, so variation across downstream seeds reflects controller training rather than skill acquisition.

4.3 Meta-Controller Training

The feed-forward meta-controller receives the complete 1345-dimensional observation. Separate actor and critic networks each use three width-128 tanh layers. The actor maps observation o_t to 24 logits $z_m(o_t)$ and defines

$$\pi_\theta(m_t = m \mid o_t) = \frac{\exp z_m(o_t)}{\sum_{m' \in \mathcal{M}} \exp z_{m'}(o_t)}, \tag{8}$$

while the critic outputs $V_\phi(o_t)$. Meta-actions are sampled during training.

When the sample is skill-backed, the corresponding feature filter and frozen policy are applied before the native environment step. A direct meta-action bypasses the library. In either case exactly one transition enters the PPO rollout buffer. The Craftax Classic reward from Equation 5 is used directly, $r_t^{\text{meta}} = r_t^{\text{env}}$; no intrinsic reward, skill bonus, or multi-step reward aggregation is added.

PPO gradients update only the actor and critic. The skill parameters remain fixed, so downstream learning can change when a behaviour is selected but not how that lower-level policy acts. This separation makes the controller comparison a test of composition rather than joint fine-tuning.

Training uses 256 parallel environments and 16-step rollouts (4096 transitions per update), Generalized Advantage Estimation with $\gamma = 0.99$ and $\lambda = 0.8$, four epochs, and eight minibatches. The clipping, entropy, and value-loss coefficients are 0.2, 0.01, and 0.5, respectively. Adam starts at 3×10^{-4} with linear annealing; gradient norm is capped at 1.0, and optimistic resets use a ratio of 16.

4.4 Recurrent PPO Architecture

The recurrent agents follow the Craftax PPO-RNN baseline [13]. A ReLU observation embedding feeds a GRU, and its output feeds separate actor and critic heads with two ReLU layers each. Counting the recurrent representation, both actor and critic paths therefore have three hidden layers. The actor ends in one logit per available action and the critic in one scalar. Flat PPO-RNN uses width 512 and 17 outputs; Meta-PPO-RNN uses width 128 and 24 outputs. The recurrent hierarchy therefore changes the controller only: all lower-level skills remain feed-forward and frozen.

Both recurrent agents use the feed-forward PPO hyperparameters and 16-step rollout segments. Hidden state persists between consecutive segments and is reset at episode termination. During optimisation, environments are shuffled between minibatches but temporal order within each 16-step sequence is preserved. The hidden state recorded at the start of collection is used when replaying that segment, allowing gradients to update the embedding, GRU, and both heads through the sequence.

4.5 Training Pipeline

Training proceeds in three stages. First, the six skills are optimised independently and their final weights are frozen. Second, the feature filters and action mappings combine those networks into eight skill-backed choices and append the 16 direct choices. Third, a newly initialised PPO controller is trained on complete, unmodified Craftax Classic maps. No downstream trajectory is used to update the skill library.

The hierarchy therefore incurs a one-time pretraining cost before controller training begins. The same frozen checkpoints are shared by Meta-PPO and Meta-PPO-RNN and across all downstream seeds. Reuse across those controllers, and later across the Extended transfer study, is relevant when interpreting the large number of pretraining transitions.

4.6 Experimental Setup and Evaluation

Four agents are compared: Flat PPO, Meta-PPO, Flat PPO-RNN, and Meta-PPO-RNN. Flat agents select among the 17 native actions with width-512 networks. Meta-controllers select among all 24 meta-actions with width-128 networks. Controller capacity is intentionally not matched: compactness is one design objective, and parameter count is reported alongside performance.

Each condition is trained from scratch with downstream seeds 0–4 at budgets from 100,000 to 20 million environment steps. Results report the mean and one sample standard deviation across those five runs. At each budget, *normalized environment return* is mean episodic return from the final PPO update, including the health term in Equation 5, divided by the 22-point maximum achievement reward and expressed as a percentage.

After 20 million steps, every checkpoint is additionally evaluated with a greedy argmax policy. For each training seed, 128 and 256 evaluation episodes are generated from master seed 42. These episodes are separate from training and use the same normalized return. Per-achievement final-update success rates and action distributions are retained as diagnostic analyses in Appendix A.

The two measurements answer different questions. Final-update return describes the policy while actions are still sampled during training and is appropriate for comparing learning curves at fixed budgets. Greedy evaluation removes sampling noise from action selection and tests fixed checkpoints on a common set of fresh episodes. It is not used to select checkpoints or tune hyperparameters. Greedy-policy contrasts are matched by training seed and reported as paired

mean differences with Student-t 95% confidence intervals and two-sided paired t-test p-values. The analysis unit is each training seed's mean over its evaluation episodes ($n = 5$, df $= 4$); episodes are not treated as independent replicates. These contrasts are exploratory, and their intervals and p-values are unadjusted for multiple comparisons.

5 Results

Figure 7 compares the four agents across five seeds. Meta-PPO has the highest observed mean return through two million steps ($36.1 \pm 3.8\%$ versus $33.0 \pm 3.3\%$ for Flat PPO), after which the ordering reverses. At 20 million steps, Meta-PPO reaches $62.2 \pm 1.5\%$ versus $64.6 \pm 1.2\%$, retaining 96.3% of Flat PPO's mean with 83.0% fewer parameters.

Meta-PPO-RNN exceeds Flat PPO-RNN at every reported budget, and the margin widens from 500,000 steps onward, reaching $58.7 \pm 2.5\%$ versus $39.0 \pm 7.7\%$ at 20 million steps. Its complete 342,063-parameter hierarchy is 89.7% smaller than Flat PPO-RNN and 86.0% smaller than feed-forward Flat PPO (Table 2). Per-achievement curves for all four agents are reported in Appendix A.8.

Table 2: Parameter counts. Complete hierarchies include the controller and all six frozen skill policies.

Model component	Parameters
Flat PPO actor–critic	2,438,162
Meta-PPO actor–critic	413,849
Frozen skill library	1,686
Complete Meta-PPO hierarchy	415,535
Flat PPO-RNN actor–critic	3,323,922
Meta-PPO-RNN actor–critic	340,377
Complete Meta-PPO-RNN hierarchy	342,063

In held-out greedy evaluation (Figure 8), Meta-PPO-RNN and feed-forward Flat PPO have the same observed mean, 58.3%, with sample standard deviations of 3.6 and 1.8 percentage points, respectively. Their paired difference is 0.04 percentage points (95% CI $[-4.01, 4.08]$, $p = 0.982$); this is numerical matching, not evidence of equivalence. Meta-PPO reaches $54.4 \pm 2.1\%$ and Flat PPO-RNN $35.2 \pm 5.4\%$. Meta-PPO-RNN and Flat PPO each exceed Flat PPO-RNN by approximately 23 points (both $p < 0.001$), whereas Flat PPO's 3.84-point advantage over Meta-PPO is less conclusive (95% CI $[-0.42, 8.09]$, $p = 0.067$). The 256-episode evaluation gives the same qualitative conclusions; full paired contrasts are reported in Appendix A.3.

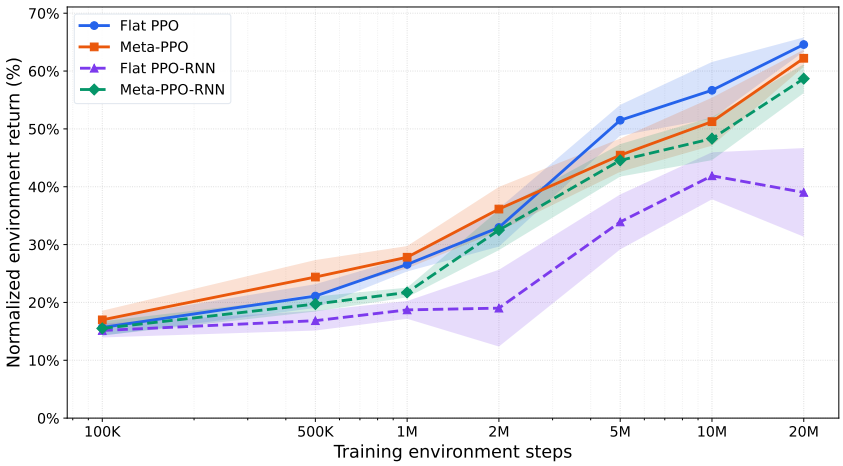


Fig. 7: Mean normalized environment return across five seeds; shading shows one sample standard deviation. Meta-PPO leads Flat PPO through two million steps, while Meta-PPO-RNN leads Flat PPO-RNN after the initial stage.

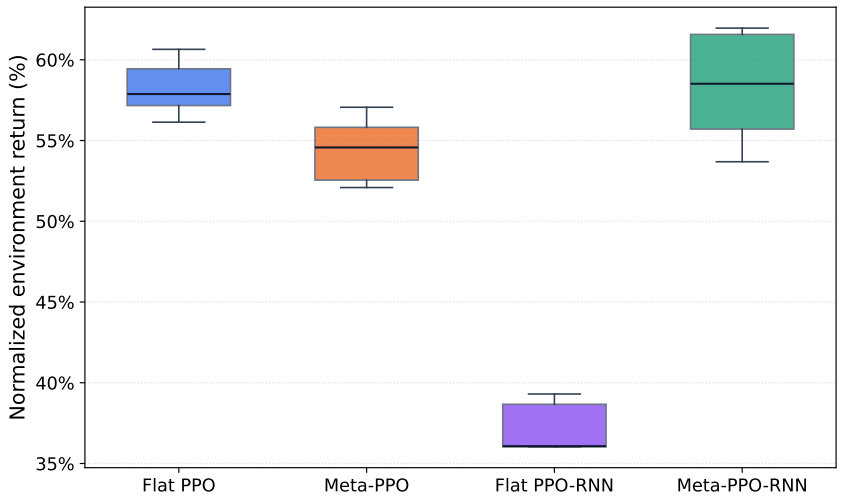


Fig. 8: Greedy evaluation after 20 million training environment steps across seeds 0–4 (128 evaluation episodes; master seed 42).

Appendix A.4 reports action distributions for the seed-4 Meta-PPO checkpoint. It selected skill-backed actions on 48.0% of greedy-evaluation steps. This result is indicative rather than an estimate across seeds.

Skill pretraining used 259.2 million environment steps, 91.3% of them for hunting and combat, but the complete six-skill library trained in under 50 minutes on the evaluation laptop because policies and bootstraps were small and vectorised (Table 1; Appendix G). On natural Classic maps, wood, food, combat, and stone achieved 64.3–94.8% unconditional success, while water and ore reached 16.6% and 7.9% (Appendix A.5).

The same checkpoints were evaluated zero-shot on matched Craftax Extended bootstraps (Figure 9). Relative to their Classic bootstrap scores (C@C), the transferred checkpoints (C@E) retain 1.05 for wood, 0.93 for stone, 1.00 for ore, 0.86 for water, 0.93 for food, and 0.63 for combat. C@E exceeds the untrained Extended baseline (U@E) for every skill. Fine-tuning and natural-map results are reported in Appendix A.6.

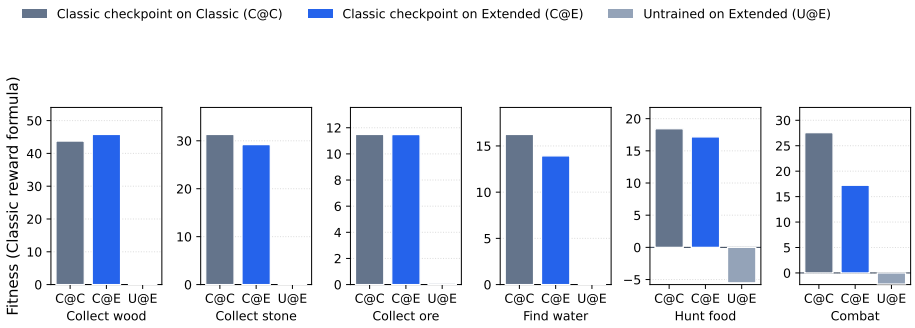


Fig. 9: Zero-shot transfer of frozen Classic skills onto matched Craftax Extended bootstraps. Bars show Classic-formula scores for Classic weights on Classic (C@C), Classic weights on Extended (C@E), and an untrained policy on Extended (U@E). The Collect Ore panel uses coal as its target.

A preliminary single-seed horizon study favours one-step control: at ten million steps, horizon one reaches 54.0% return versus 36.2% for horizon four (Appendix A.7).

6 Discussion

6.1 Downstream Performance and the Skill Interface

Frozen skills support one-step downstream control and improve the observed mean through two million steps, but not at the largest budget. Because standard-deviation bands overlap at several checkpoints, this should be read as a trend

rather than as conclusive evidence. The final greedy comparison likewise estimates Flat PPO's advantage over Meta-PPO at approximately four percentage points, but its 95% confidence interval includes zero at both evaluation budgets. With five runs, this is insufficient evidence of a non-zero difference rather than evidence that the methods are equivalent. The per-achievement results suggest that the early gain is concentrated in interaction-heavy milestones: the controller begins with useful local behaviours instead of discovering movement–interaction patterns only through sparse achievement reward.

A flat policy must learn both when interaction is useful and which movements and interactions realise the current objective. A skill-backed action already implements a local tendency such as approaching stone or attacking a nearby monster. Withholding direct `DO` further turns those tendencies into the only route to resource collection and combat. Early in training, an imperfect but relevant skill may therefore reach reward more readily than an unstructured primitive-action policy.

The same interface possibly explains the later reversal. Fixed feature vectors omit most of the full observation, and the skills cannot adapt when that missing context changes the appropriate interaction. Flat PPO can eventually refine one end-to-end mapping to every native action, whereas Meta-PPO remains constrained by its library. The seed-4 Meta-PPO checkpoint nevertheless selects skill-backed actions on 48.0% of greedy-evaluation steps (Appendix A.4), which suggests that its result reflects actual use of the hierarchy rather than avoidance through direct actions.

The feed-forward crossover is not gradual: Meta-PPO leads by 3.1 percentage points at two million steps, but trails by 6.0 points at five million. This pattern is consistent with a strong initial bias followed by a ceiling imposed by the fixed interface. The action-distribution result also makes unlikely a simple explanation in which the controller learns to ignore its frozen library, although it is measured on a single seed. However, frequency is not contribution: a common skill may repeat low-impact movement, while a rare interaction may unlock an achievement chain. Ablation experiments could be useful in isolating the contribution of individual skills.

6.2 Recurrent Performance and Parameter Efficiency

The recurrent comparison strengthens the parameter-efficiency result. Meta-PPO-RNN uses 342,063 parameters including all skills, versus 3,323,922 for Flat PPO-RNN, yet has a much higher final training return. Its paired greedy advantage over Flat PPO-RNN is approximately 23 percentage points and its 95% confidence interval excludes zero at both evaluation budgets. More notably, its held-out greedy mean matches that of feed-forward Flat PPO at 58.3%, despite using 86.0% fewer parameters. However, the wide confidence interval around the near-zero paired difference means that modest performance differences in either direction cannot be ruled out. The architectures also differ in width and output space, and Flat PPO-RNN declines after ten million steps, so the result cannot

isolate hierarchy from capacity or optimisation stability. A possible interpretation is that skills offload local interaction patterns while the smaller GRU tracks longer-term context.

The recurrent learning curves also behave differently. Meta-PPO-RNN improves monotonically across all reported budgets, whereas Flat PPO-RNN peaks at ten million steps and then declines from 41.9% to 39.0%. The 19.7-point final gap therefore combines continued hierarchical learning with deterioration in the recurrent flat baseline and should not be attributed to the hierarchy alone.

The held-out comparison adds a second perspective because the two 58.3% means arise from different training trajectories. Flat PPO has the stronger final-update training return, while Meta-PPO-RNN reaches the same greedy evaluation mean from a lower final training mean. Differences in stochastic action selection, final-update episode composition, or robustness across fresh layouts could contribute. Five checkpoints per condition cannot separate these explanations, but the result shows that the compact recurrent hierarchy reaches the observed evaluation level of the much larger feed-forward reference, not only that it outperforms Flat PPO-RNN.

Feed-forward Meta-PPO similarly retains 96.3% of Flat PPO's final mean with 83.0% fewer parameters. These are parameter-efficiency results, not total sample-efficiency results: constructing the shared skill library required 259.2 million pretraining steps, 91.3% of which were devoted to food and combat. This one-time cost is shared across both hierarchical controllers and all downstream seeds. Vectorisation makes these transitions cheap in wall-clock time, but pretraining is most defensible when the library is reused across controllers or tasks.

The uneven budget reflects task structure as well as algorithmic efficiency. Food and combat use longer horizons and moving-target curricula, while static resources can be learned in shorter rollouts. The recorded budget is the cost of the chosen schedules, not a demonstrated minimum. On the hardware reported in Appendix G, the complete six-skill library nevertheless trained in under 50 minutes, illustrating why environment-step and wall-clock efficiency should be reported separately for vectorised neuroevolution.

6.3 Skill Robustness, Transfer, and Temporal Abstraction

Natural-map results (Figure 14) show that wood, food, combat, and stone generalise beyond their bootstraps, whereas water and ore are less reliable and several gathering skills have high death rates. High success conditional on encountering a target confirms local competence, while death and low encounter rates show why a controller is still required for survival and sequencing.

Transfer to Extended is likewise selective (Figure 9). The C@C, C@E, and U@E comparison shows that the transferred weights outperform untrained policies for every matched task. Five skills retain at least 86% of their Classic bootstrap score, while combat retains 63%. This provides direct evidence of zero-shot skill reuse, although the reduced combat score shows that the larger observation and changed environment still matter. Short fine-tuning helps water, stone, and ore on natural maps but slightly hurts food and combat (Figures 16 and 17).

Since the same fine-tune phase improves their bootstrap objectives, the mismatch cautions against treating adaptation on injected layouts as general transfer. Under matched budgets, warm-starting Extended OpenES from Classic weights also provides a large advantage over training from scratch, particularly for *Hunt Food* and *Combat*; these were the two most expensive Classic skills to pretrain, so reuse preserves the costliest acquired behaviours (Figure 15). The single matched seed makes this warm-start advantage a hypothesis rather than a definitive sample-efficiency claim.

Classic combat also achieves similar observed kill rates on unseen Extended melee types despite their higher health, indicating that the learned behaviour transfers beyond enemies used for training (Figure 18).

The natural-map evaluation asks a broader question than the matched bootstrap comparison because success depends on both encountering a target and acting competently once it appears. Wood is common and locally simple, so its near-identical Classic and Extended success is strong evidence of direct reuse. Water and ore combine lower encounter rates with longer approach or interaction sequences; their low zero-shot rates do not isolate perception from control failure. The opposing fine-tuning effects are therefore useful: improvement on water and stone shows that the transferred weights remain adaptable, whereas deterioration on food and combat shows that optimising an injected training layout can reduce performance under the natural distribution.

The horizon study (Figure 19) suggests why one-step composition matters. Although a selected skill recomputes its native action from every new observation, the controller cannot change which skill is active until the fixed horizon ends. A four-step commitment therefore delays switching after a target disappears, a resource is collected, an enemy moves, or a survival need becomes urgent; horizon one performs substantially better. This conclusion remains preliminary because the study uses one seed and does not compare learned termination.

7 Conclusion, Limitations, and Future Work

Frozen neuroevolved skills provide useful behavioural abstraction without the explicit need for temporal commitment. Meta-PPO improves the observed mean at smaller downstream budgets and approaches Flat PPO's final return with far fewer parameters; Meta-PPO-RNN also numerically matches the held-out greedy return of feed-forward Flat PPO with 86.0% fewer parameters. The hierarchy does not improve final feed-forward training return and requires substantial specialised pretraining. Its contribution is therefore compact, modular control rather than an unqualified performance gain.

The research question can consequently be answered in three parts. At small downstream budgets, the frozen library provides a useful bias: Meta-PPO has the higher observed mean through two million steps, and achievement diagnostics suggest that the early advantage is concentrated in interaction-heavy behaviour. At larger feed-forward budgets, Flat PPO overtakes the hierarchy, suggesting that an end-to-end policy eventually benefits from adapting its local

actions to the full observation. With recurrence, however, the compact hierarchy remains substantially stronger than Flat PPO-RNN throughout later training and reaches the same observed held-out mean as feed-forward Flat PPO. The combined evidence favours the hierarchy most clearly when compactness and early reusable competence matter, rather than when final feed-forward return is the only objective.

The supplementary studies clarify what is being reused. In a single-seed comparison, one-step invocation outperforms a four-step fixed horizon, suggesting that the benefit does not require conventional temporal abstraction. Natural-map and Extended evaluations show that several frozen skills retain competence outside their bootstraps, but failures on water and ore demonstrate that local policies do not replace exploration, survival management, or high-level sequencing. Transfer is therefore behavioural and selective: the library supplies useful local tendencies, while the controller remains responsible for deciding when those tendencies match the current state.

The main comparison uses five seeds, is restricted to Classic, reports training return except for the final greedy evaluation, and is not capacity-controlled. The hierarchical agents also require 259.2 million skill-pretraining environment steps that the downstream budgets do not include, so the parameter-efficiency results reported here are not sample-efficiency results. Extended fine-tuning and temporal abstraction use one training seed. Hand-designed fitness functions, curricula, feature filters, and bootstraps mean the experiments evaluate the engineered system rather than neuroevolution in isolation; the single-machine timing is also hardware-dependent.

Several confounds follow from the architecture. Flat and hierarchical agents differ simultaneously in width, output space, and access to direct `DO`; the recurrent comparison additionally inherits the late decline of Flat PPO-RNN. The skill library is shared across hierarchical seeds, so those runs measure variation in controller training but not variation in skill acquisition. Normalized return also divides by the 22 achievement points while retaining health-change reward, meaning values are useful for within-protocol comparison but are not bounded achievement percentages. Finally, a single master seed generates the held-out evaluation batches, which improves comparability but samples only one finite set of procedural worlds.

The most important follow-up is a study that separately controls controller width, direct `DO` access, recurrence, and the presence of pretrained skills. Repeating skill acquisition across multiple seeds could capture how variability in the learned skill library affects downstream performance, while leave-one-skill-out ablations could reveal which behaviours create the early advantage. Reporting confidence intervals over independently generated evaluation batches would strengthen the numerical comparison between Meta-PPO-RNN and Flat PPO.

Taken together, these results position frozen neuroevolved skills not as a universal replacement for end-to-end learning, but as a compact and reusable basis for hierarchical control whose value depends on the task, training budget, and skill interface.

References

1. Bacon, P.L., Harb, J., Precup, D.: The option-critic architecture. In: Proceedings of the AAAI conference on artificial intelligence. vol. 31 (2017)
2. Chalumeau, F., Boige, R., Lim, B., Macé, V., Allard, M., Flajolet, A., Cully, A., Pierrot, T.: Neuroevolution is a competitive alternative to reinforcement learning for skill discovery (2023), https://arxiv.org/abs/2210.03516
3. Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using RNN encoder-decoder for statistical machine translation (2014), https://arxiv.org/abs/1406.1078
4. Dedieu, A., Ortiz, J., Lou, X., Wendelken, C., Lehrach, W., Guntupalli, J.S., Lazaro-Gredilla, M., Murphy, K.P.: Improving transformer world models for data-efficient RL (2025), https://arxiv.org/abs/2502.01591
5. Eysenbach, B., Gupta, A., Ibarz, J., Levine, S.: Diversity is all you need: Learning skills without a reward function (2018), https://arxiv.org/abs/1802.06070
6. Hafner, D.: Benchmarking the spectrum of agent capabilities (2022), https://arxiv.org/abs/2109.06780
7. Hafner, D., Pasukonis, J., Ba, J., Lillicrap, T.: Mastering diverse domains through world models (2024), https://arxiv.org/abs/2301.04104
8. Hausknecht, M., Stone, P.: Deep recurrent Q-Learning for partially observable MDPs. In: 2015 AAAI Fall Symposium Series (2015), https://arxiv.org/abs/1507.06527
9. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997). https://doi.org/10.1162/neco.1997.9.8.1735
10. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization (2017), https://arxiv.org/abs/1412.6980
11. Kulkarni, T.D., Narasimhan, K., Saeedi, A., Tenenbaum, J.: Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In: Lee, D., Sugiyama, M., Luxburg, U., Guyon, I., Garnett, R. (eds.) Advances in Neural Information Processing Systems. vol. 29. Curran Associates, Inc. (2016), https://proceedings.neurips.cc/paper_files/paper/2016/file/f442d33fa06832082290ad8544a8da27-Paper.pdf
12. Lange, R.T.: evosax: JAX-based evolution strategies (2022), https://arxiv.org/abs/2212.04180
13. Matthews, M., Beukman, M., Ellis, B., Samvelyan, M., Jackson, M., Coward, S., Foerster, J.: Craftax: A lightning-fast benchmark for open-ended reinforcement learning (2024), https://arxiv.org/abs/2402.16801
14. Matthews, M., Samvelyan, M., Parker-Holder, J., Grefenstette, E., Rocktäschel, T.: Hierarchical kickstarting for skill transfer in reinforcement learning. In: Proceedings of the 1st Conference on Lifelong Learning Agents. Proceedings of Machine Learning Research, vol. 199, pp. 856–874. PMLR (2022), https://proceedings.mlr.press/v199/matthews22a.html
15. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015). https://doi.org/10.1038/nature14236
16. Moon, S., Yeom, J., Park, B., Song, H.O.: Discovering hierarchical achievements in reinforcement learning via contrastive learning (2023), https://arxiv.org/abs/2307.03486

17. Nachum, O., Tang, H., Lu, X., Gu, S., Lee, H., Levine, S.: Why does hierarchy (sometimes) work so well in reinforcement learning? (2019), https://arxiv.org/abs/1909.10618
18. Salimans, T., Ho, J., Chen, X., Sidor, S., Sutskever, I.: Evolution strategies as a scalable alternative to reinforcement learning (2017), https://arxiv.org/abs/1703.03864
19. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms (2017), https://arxiv.org/abs/1707.06347
20. Such, F.P., Madhavan, V., Conti, E., Lehman, J., Stanley, K.O., Clune, J.: Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. arXiv preprint arXiv:1712.06567 (2017). https://doi.org/10.48550/arXiv.1712.06567
21. Sutton, R.S., Precup, D., Singh, S.: Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence **112**(1), 181–211 (1999). https://doi.org/10.1016/S0004-3702(99)00052-1, https://www.sciencedirect.com/science/article/pii/S0004370299000521
22. Vezhnevets, A.S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., Kavukcuoglu, K.: FeUdal networks for hierarchical reinforcement learning. In: Proceedings of the 34th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 70, pp. 3540–3549. PMLR (2017), https://proceedings.mlr.press/v70/vezhnevets17a.html
23. Wierstra, D., Schaul, T., Glasmachers, T., Sun, Y., Schmidhuber, J.: Natural evolution strategies (2011), https://arxiv.org/abs/1106.4487

A Additional Experimental Results

A.1 Feed-Forward PPO

Table 3: Normalized environment return for the feed-forward agents (%; mean ± one sample standard deviation over seeds 0–4).

Env. steps	Flat PPO	Meta-PPO
100k	15.7±1.4	**17.0±1.6**
500k	21.1±2.1	**24.4±2.9**
1m	26.6±1.2	**27.8±1.9**
2m	33.0±3.3	**36.1±3.8**
5m	**51.5±2.7**	45.5±2.9
10m	**56.7±4.9**	51.2±4.1
20m	**64.6±1.2**	62.2±1.5

A.2 PPO-RNN

Table 4: Normalized environment return for the recurrent agents (%; mean ± one sample standard deviation over seeds 0–4).

Env. steps	Flat PPO-RNN	Meta-PPO-RNN
100k	15.2±1.2	**15.5±1.2**
500k	16.8±1.7	**19.7±1.3**
1m	18.7±1.5	**21.7±0.8**
2m	19.0±6.6	**32.5±3.5**
5m	33.9±4.7	**44.6±2.8**
10m	41.9±4.1	**48.3±3.7**
20m	39.0±7.7	**58.7±2.5**

A.3 Greedy Evaluation

Table 5: Greedy evaluation after 20 million training environment steps (normalized environment return in %; mean ± one sample standard deviation over training seeds 0–4). Evaluation uses master seed 42 and an argmax policy.

Episodes	Flat PPO	Meta-PPO	Flat PPO-RNN	Meta-PPO-RNN
128	**58.3±1.8**	54.4±2.1	35.2±5.4	**58.3±3.6**
256	**58.5±1.7**	54.5±1.7	35.3±5.7	58.4±3.7

Table 6 reports the corresponding seed-matched policy contrasts.

Table 6: Paired mean differences in greedy normalized environment return (percentage points) after twenty million training steps. Each contrast is matched on training seeds 0–4; evaluation uses master seed 42. Cells report mean difference with a Student-t 95% CI and two-sided paired t-test p-value (n=5, df=4). The analysis unit is the per-seed mean over evaluation episodes. Intervals and p-values are unadjusted for multiple comparisons.

Contrast	128 episodes	256 episodes
Flat − Meta	+3.84 [−0.42, 8.09] (p=0.067)	+3.99 [−0.03, 8.00] (p=0.051)
Meta-RNN − Flat	+0.04 [−4.01, 4.08] (p=0.982)	−0.06 [−4.69, 4.56] (p=0.973)
Meta-RNN − Flat-RNN	+23.10 [17.16, 29.04] (p<0.001)	+23.05 [16.82, 29.27] (p<0.001)
Flat − Flat-RNN	+23.07 [15.98, 30.16] (p<0.001)	+23.11 [15.13, 31.09] (p=0.001)
Meta-RNN − Meta	+3.87 [−0.60, 8.34] (p=0.074)	+3.93 [−0.62, 8.48] (p=0.075)

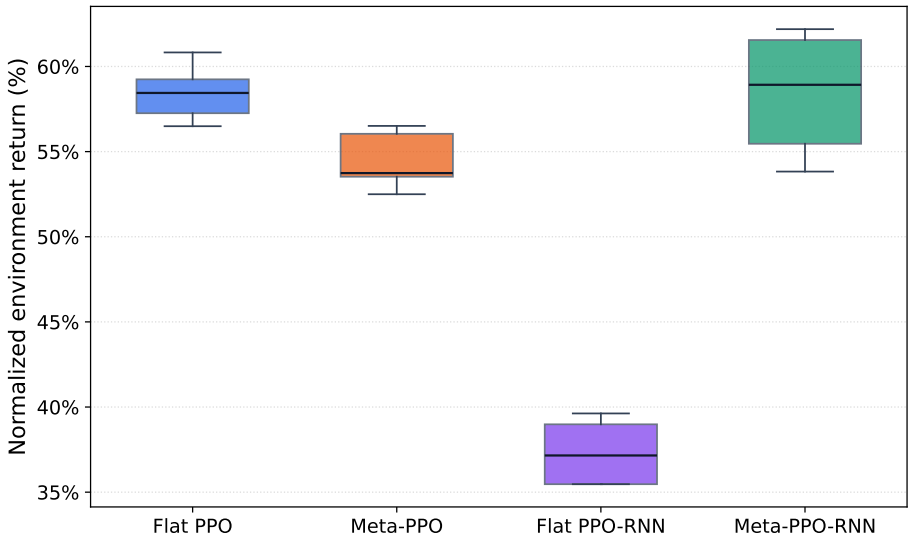
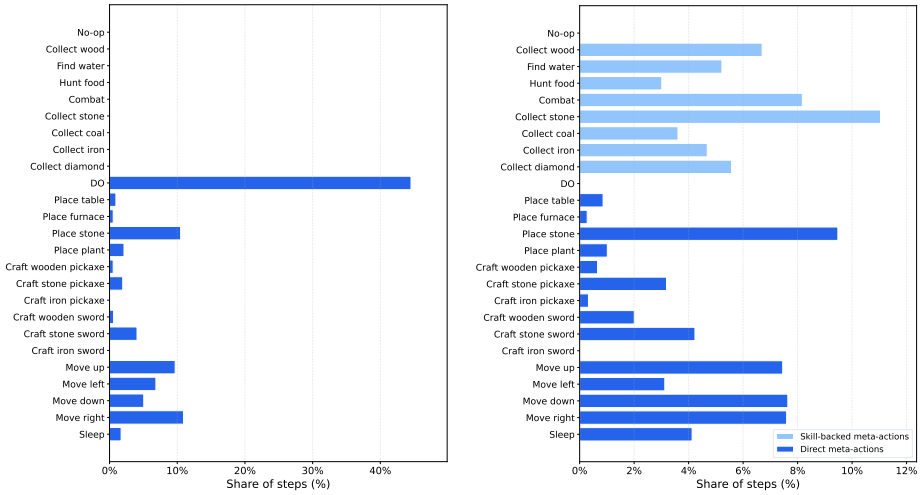


Fig. 10: Greedy evaluation after 20 million training environment steps across seeds 0–4 (256 evaluation episodes; master seed 42). Doubling the evaluation budget relative to Figure 8 leaves the ranking unchanged.

A.4 Action Distributions

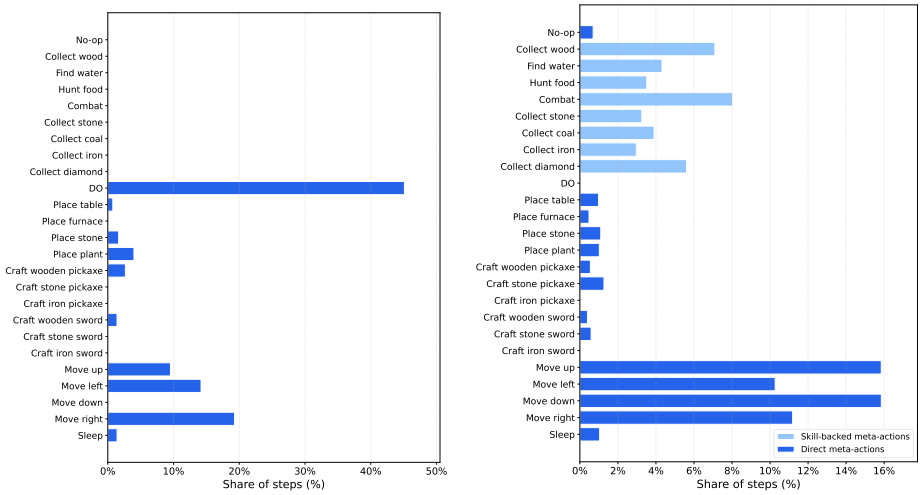
All percentages in this subsection are computed from the training-seed-4 checkpoint of each agent, not averaged across the five training seeds. Across 128 greedy evaluation episodes, Flat PPO selected `DO` on 44.5% of steps. Meta-PPO selected skill-backed meta-actions on 48.0% of steps, most often *Collect Stone* (11.0%) and *Combat* (8.2%).

Figure 12 reports the greedy action distributions for the Flat PPO-RNN and Meta-PPO-RNN training-seed-4 checkpoints under the same evaluation protocol as Figure 11. Flat PPO-RNN selected `DO` on 45.1% of steps. Meta-PPO-RNN selected skill-backed meta-actions on 38.7% of steps, with movement and sleep accounting for the majority of the remaining mass.



(a) Flat PPO (b) Meta-PPO

Fig. 11: Greedy action distributions for the Flat PPO and Meta-PPO training-seed-4 checkpoints after 20 million training steps, aggregated over 128 evaluation episodes from master seed 42.



(a) Flat PPO-RNN (b) Meta-PPO-RNN

Fig. 12: Greedy action distributions for the Flat PPO-RNN and Meta-PPO-RNN training-seed-4 checkpoints after 20 million training environment steps, aggregated over 128 evaluation episodes generated from master seed 42. Layout and colour coding match Figure 11.

A.5 Lower-Level Skill Training Results

This appendix reports OpenES training outcomes for the six frozen skills used by Meta-PPO. All statistics are taken from the exact checkpoints referenced by the Meta-PPO run configurations. Fitness and diagnostic rates are measured in the corresponding bootstrapped skill environments during the final OpenES generation; they are not evaluations on unmodified Craftax Classic maps.

Table 7: Final OpenES population statistics for the frozen lower-level skills used by Meta-PPO. Mean and best fitness are taken from the last generation of the final training phase. The diagnostic rate is the corresponding last-generation training metric on the bootstrapped skill environment (reached tree, drink success, cow kill rate, hostile kill rate, or mined rate).

Skill	Phases	Gens.	Mean fitness	Best fitness	Final diagnostic
Collect Wood	1	100	39.73	42.59	Reached tree: 99.4%
Find Water	1	100	26.46	27.00	Drink success: 29.4%
Hunt Food	2	200	18.43	19.20	Cow kill rate: 83.9%
Combat	2	200	30.34	31.72	Kill rate: 87.4%
Collect Stone	1	100	30.06	33.80	Mined rate: 80.4%
Collect Ore	1	200	11.43	11.43	Mined rate: 100.0%

The resource-gathering skills converge to high bootstrap competence: the wood skill reaches a visible tree on 99.4% of final-generation rollouts, the stone skill mines stone on 80.4%, and the ore skill mines its curriculum target on every final-generation rollout. *Hunt Food* and *Combat* retain high kill rates after the moving-target phase (83.9% and 87.4%, respectively). *Find Water* attains high fitness through cumulative drink gain, while its stricter drink-success diagnostic remains lower (29.4%), reflecting the gap between approaching water and successfully drinking within the short rollout horizon.

Natural-Map Transfer To test whether the frozen skills remain useful outside their bootstrapped training environments, each skill was evaluated alone on naturally generated Craftax Classic maps without artificial target injection. The procedural terrain and target layouts were retained, while task prerequisites were staged by emptying relevant meters or inventories, supplying required tools, and removing hostiles for the mining skills. Evaluations used greedy skill actions in 1024 rollouts generated from master seed 0, with a horizon of 128 steps. Success is a minimal task-completion criterion (for example, collecting at least one wood), not a fitness score. Table 8 and Figure 14 report unconditional success, success conditioned on ever seeing the target, encounter rates, and death rates.

Collect Wood, *Hunt Food*, *Combat*, and *Collect Stone* transfer reasonably once a target appears (success given encounter between 69% and 95%). *Find Water* is weaker: it sees water on 65.8% of rollouts but succeeds on only 25.2%



Fig. 13: OpenES training curves for the six lower-level skills. Solid lines show the population mean fitness and shaded bands show ± one population standard deviation; dashed lines show the best fitness in the population. Vertical dotted lines mark the transition between curriculum phases for *Hunt Food* and *Combat*. Generations are indexed continuously across phases.

Table 8: Natural-map transfer of the frozen Meta-PPO skills on naturally generated Craftax Classic maps without artificial target injection (greedy skill-only evaluation; N=1024 rollouts generated from master seed 0; horizon 128 steps). *Ever saw* is the fraction of rollouts in which the skill target entered the local field of view at least once. *Success* is the unconditional goal rate. *Success* | *ever* conditions on ever seeing the target. *Death* is the fraction of rollouts ending with zero health. Source JSON: `runs/benchmark_reports/skill_natural_eval.json`.

Skill	Ever saw (%)	Success (%)	Success | ever (%)	Death (%)	Success rule
Collect Wood	99.7	94.8	95.1	37.8	Δwood ≥ 1
Find Water	65.8	16.6	25.2	52.5	Δdrink > 0
Hunt Food	96.5	89.4	92.6	39.8	≥ 1 cow kill or Δfood > 0
Combat	74.7	64.3	86.0	2.1	≥ 1 hostile kill
Collect Stone	92.8	64.4	69.4	36.4	Δstone ≥ 1
Collect Ore	47.3	7.9	13.8	33.6	Δcoal or Δiron ≥ 1

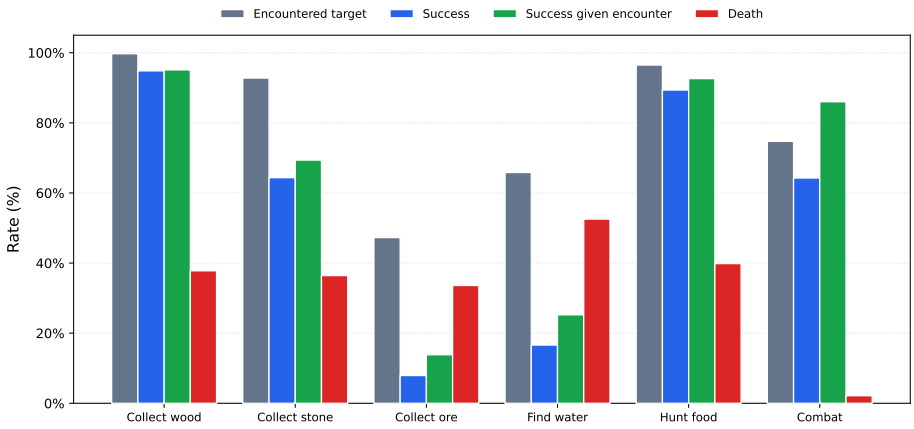


Fig. 14: Natural-map transfer rates for the frozen Meta-PPO skills on naturally generated Craftax Classic maps without artificial target injection (N=1024 rollouts generated from master seed 0, horizon 128). Grouped bars show the fraction of rollouts that ever saw the skill target, succeeded unconditionally, succeeded given that the target was seen, and ended in death.

of those encounters. *Collect Ore* remains the weakest transfer case, with both low encounter rates and low conditional mining success. High death rates on several gathering skills further indicate that a meta-controller is still needed for survival, even when an individual skill can complete its local objective.

A.6 Transfer to Craftax Extended

This appendix evaluates whether the frozen Classic skill library transfers to Craftax Extended. Skills are matched to Extended through the same skill-specific feature filters used on Classic, applied to the larger Extended symbolic observation. Unless noted otherwise, food and combat bootstrap numbers use the corrected mob-relative feature slices. Bootstrap zero-shot scores use the Classic reward formula on both Classic and Extended so that C@C and C@E share one scale.

Bootstrap Transfer Each Classic skill checkpoint is evaluated greedily on its Classic bootstrap environment (C@C) and on the corresponding Extended bootstrap (C@E), alongside an untrained policy on the Extended bootstrap (U@E). Table 9 and Figure 9 summarise the results.

Gathering skills retain essentially all of their Classic bootstrap competence: wood retention is 1.05, stone 0.93, and ore (coal, wall depth 2) 1.00. Water retains 0.86 of its Classic score, food retains 0.93, and combat retains 0.63 (27.6 $\rightarrow$ 17.2).

Table 9: Zero-shot transfer of frozen Classic skills onto bootstrapped Craftax Extended environments, scored with the Classic reward formula on both environments so C@C and C@E share one scale. C@C: Classic checkpoint on Classic; C@E: Classic checkpoint on Extended; U@E: untrained policy on Extended. Retention is C@E/C@C. Ore row is coal at wall depth 2.

Skill	Metric	C@C	C@E	U@E	Retention
Collect Wood	score	43.76	45.77	0.01	1.05
Collect Stone	score	31.32	29.20	0.01	0.93
Collect Ore	score	11.49	11.48	0.11	1.00
Find Water	score	16.24	13.92	0.00	0.86
Hunt Food	score	18.43	17.16	-5.51	0.93
Combat	score	27.56	17.23	-2.18	0.63

Fine-Tuning Versus Training from Scratch For all six skills, OpenES was run for fifty generations on an Extended bootstrap from either a random initialisation or a Classic warm start (matched seed, population, and rollout budget). Table 10 and Figure 15 show higher final fitness from Classic initialisation than from scratch for this preliminary matched-seed comparison. The gap is largest for water, food, combat, stone, and ore; wood improves more modestly because scratch already reaches high fitness within fifty generations.

Table 10: Matched OpenES budgets on Craftax Extended bootstrap (seed 42; pop-size 100; 64 parallel rollouts). Fine-tuning warm-starts the search mean from Classic skill-policy checkpoints.

Skill	Scratch	Classic fine-tune
Collect Wood	39.4	**43.5**
Collect Stone	12.7	**33.0**
Collect Ore	5.5	**11.3**
Find Water	3.1	**62.7**
Hunt Food	43.2	**86.4**
Combat	5.9	**23.0**

Natural-Map Transfer Following the Classic protocol in Appendix A.5, each skill was evaluated alone on naturally generated Extended maps without artificial target injection, with limited prerequisite staging (N=1024, horizon 128, master seed 0). Table 11, Figure 16, and Figure 17 report the rates.

Comparing Figure 16 and Figure 17 shows that the effects of fine-tuning extend beyond unconditional success. Wood transfers almost identically to Classic natural success (94.8%), and fifty-generation fine-tuning leaves it essentially unchanged (94.7%). Stone and ore improve modestly under fine-tuning (52.7% →

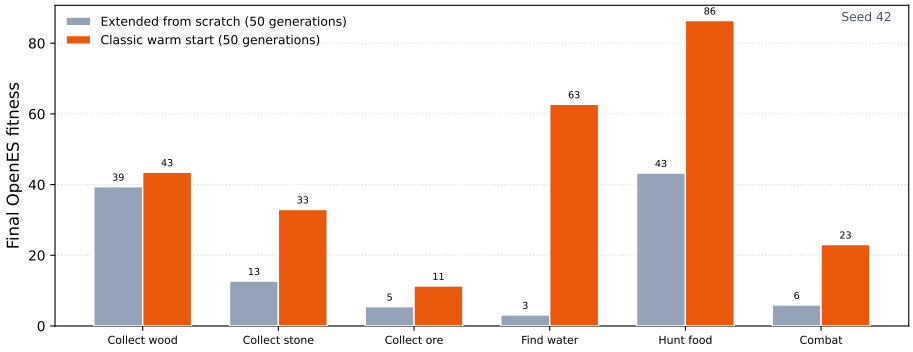


Fig. 15: Matched-budget OpenES on Craftax Extended bootstrap: training from scratch versus fine-tuning Classic skill-policy checkpoints for fifty generations (all six skills; single matched seed, seed 42).

Table 11: Natural-map evaluation on naturally generated Craftax Extended maps without artificial target injection and with limited prerequisite staging (greedy skill-only rollouts; N=1024; horizon 128; master seed 0). Columns mirror the Classic natural protocol. FT50 success uses fifty-generation Classic→Extended fine-tunes for all six skills. Classic natural is the corresponding Classic-map success from Table 8.

Skill	Ever saw (%)	Success (%)	Success | ever (%)	Death (%)	FT50 success (%)	Classic natural (%)
Collect Wood	99.7	94.8	95.1	9.6	94.7	94.8
Collect Stone	80.1	52.7	65.9	8.3	58.6	64.4
Collect Ore	43.3	5.7	12.6	12.8	6.5	7.9
Find Water	58.6	19.3	33.0	21.4	40.3	16.6
Hunt Food	94.3	81.3	86.2	12.0	77.9	89.4
Combat	14.3	12.6	83.6	0.5	10.5	64.3

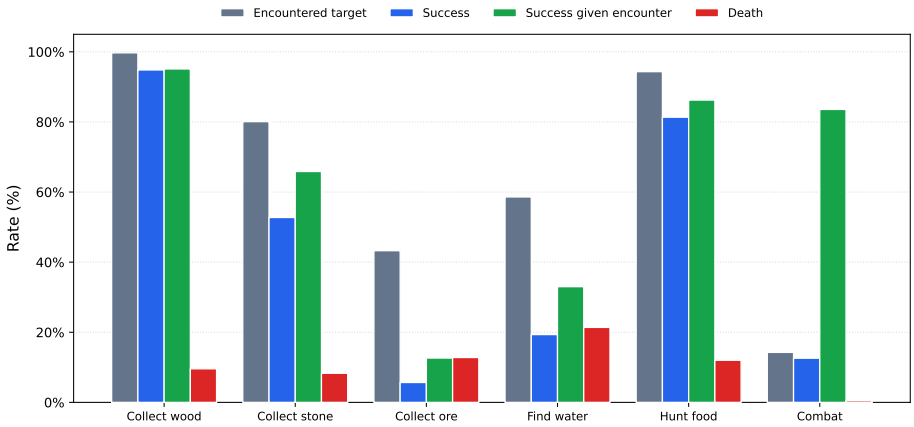


Fig. 16: Natural Extended transfer rates for Classic skill-policy checkpoints evaluated zero-shot (greedy skill-only rollouts; N=1024; horizon 128). Grouped bars show ever-saw, unconditional success, success given encounter, and death, matching Figure 14.

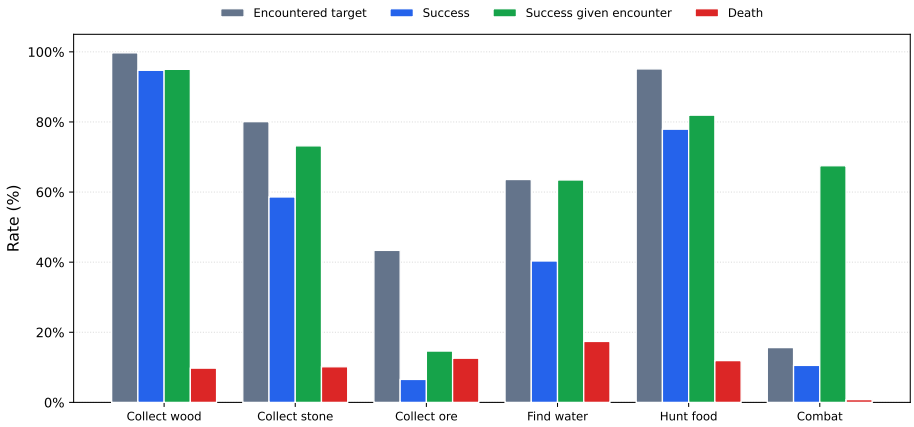


Fig. 17: Natural Extended transfer rates after fifty generations of fine-tuning the Classic skill-policy checkpoints on Extended (greedy skill-only rollouts; N=1024; horizon 128). Grouped bars match Figure 16.

58.6% and 5.7% → 6.5%). Water improves substantially (19.3% → 40.3%); its success given encounter also increases considerably while its encounter rate changes only modestly, indicating improved task execution rather than merely greater target availability. Food and combat instead lose a little natural success after the same fine-tuning procedure (81.3% → 77.9% and 12.6% → 10.5%), even though both raise bootstrap fitness in Table 10. The fine-tune objective is OpenES fitness on Extended bootstrap layouts with injected targets; natural success measures binary completion on naturally generated maps under the staged protocol, without target injection. Food's drop is small and mainly reduces success given encounter (86.2% → 81.9%). Combat remains encounter-limited (ever-saw ≈ 14–16% versus ≈ 75% on Classic natural), but success given encounter falls more sharply (83.6% → 67.5%), indicating that bootstrap adaptation can hurt competence on sparse natural fights even when it improves the training fitness.

Novel Melee Transfer Classic combat was evaluated zero-shot against Extended bootstrap melee types whose identities are not exposed in the skill observation (melee channels are summed). Table 12 and Figure 18 show essentially equal kill rates on zombie, gnome warrior, and orc soldier, all far above the untrained baseline.

Table 12: Zero-shot Classic combat on Extended bootstrap melee types. The observation sums melee channels and does not expose type identity; gnome warrior and orc soldier appear only in Craftax Extended.

Melee type	HP	Classic kill rate	Untrained	vs zombie
zombie	5	0.338	0.000	1.00
gnome warrior	7	0.341	0.000	1.01
orc soldier	9	0.347	0.000	1.03

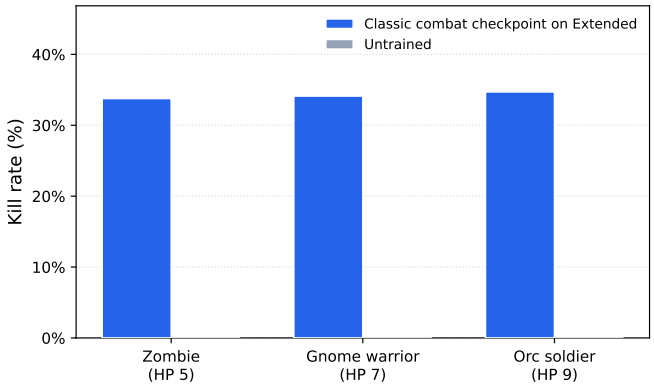


Fig. 18: Zero-shot Classic combat kill rates on Extended-only melee types (gnome warrior, orc soldier) versus zombie and an untrained baseline. Mean over five seeds; observations do not encode melee type identity.

A.7 Fixed-Horizon Comparison

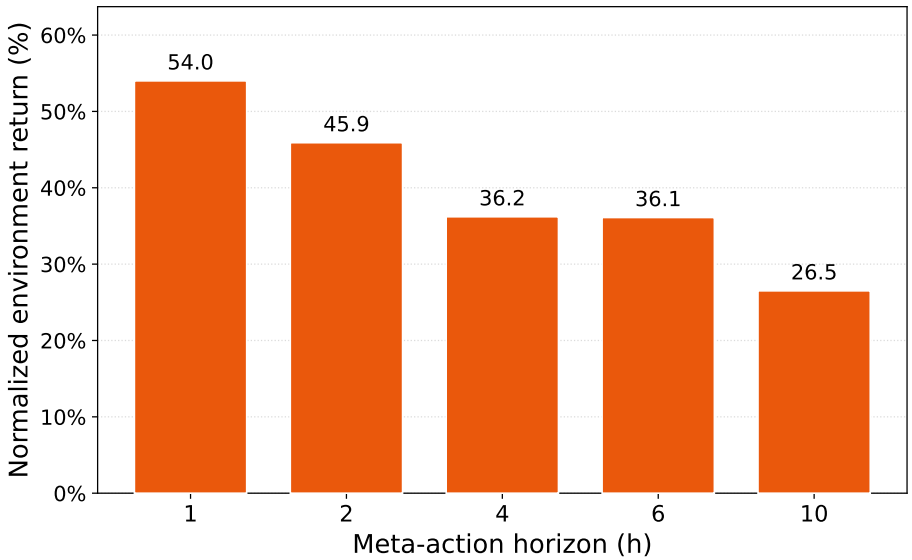
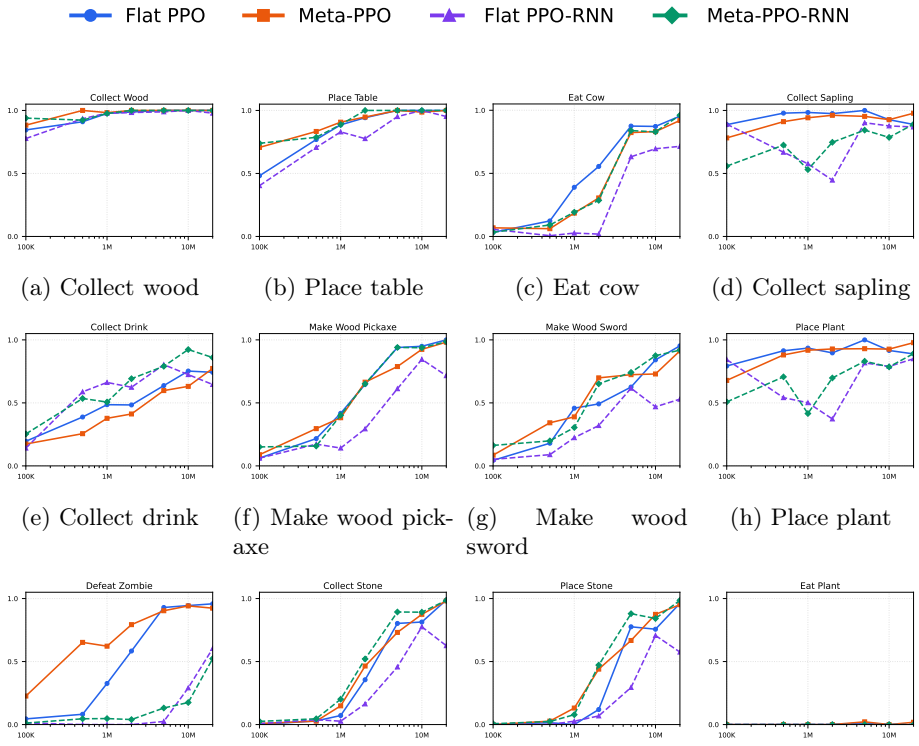


Fig. 19: Effect of fixed meta-action horizon h on Meta-PPO after ten million environment steps on Craftax Classic (seed 42). The selected meta-action is held for h steps: skill-backed meta-actions recompute a native Craftax action at every step, whereas direct meta-actions repeat their fixed native action.

A.8 Per-Achievement Training Success Rates

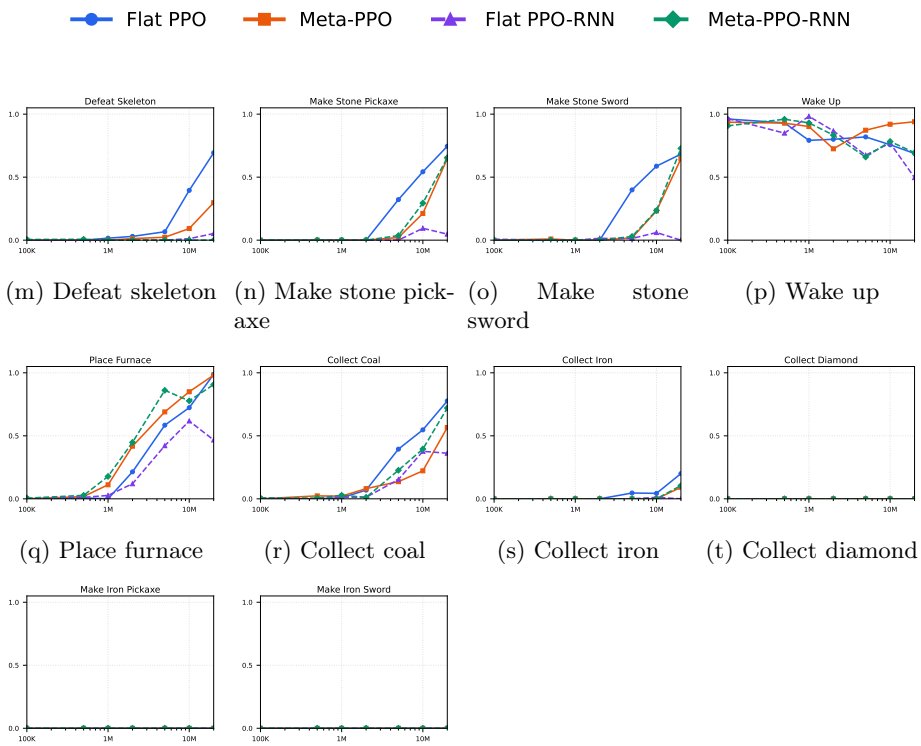


(a) Collect wood (b) Place table (c) Eat cow (d) Collect sapling

(e) Collect drink (f) Make wood pick-axe (g) Make wood sword (h) Place plant

(i) Defeat zombie (j) Collect stone (k) Place stone (l) Eat plant

Fig. 20: Per-achievement training success rates on Craftax Classic (part 1 of 2). Each panel shows the mean last-update success rate across training seeds 0–4 for Flat PPO, Meta-PPO, Flat PPO-RNN, and Meta-PPO-RNN at fixed environment-step budgets from 100k to 20M (log-scaled x-axis).



(m) Defeat skeleton (n) Make stone pickaxe (o) Make stone sword (p) Wake up

(q) Place furnace (r) Collect coal (s) Collect iron (t) Collect diamond

(u) Make iron pickaxe (v) Make iron sword

Fig. 20: Per-achievement training success rates on Craftax Classic (part 2 of 2). Axes and policies match Figure 20.

The feed-forward curves suggest that Meta-PPO's early advantage was concentrated in a subset of interaction-heavy milestones. At two million steps, Meta-PPO led Flat PPO by 32.1 percentage points on *Place Stone* and by approximately 20 percentage points on *Defeat Zombie*, *Make Wood Sword*, and *Place Furnace*, while trailing on *Eat Cow* and *Collect Drink*. By 20 million steps, Flat PPO led most clearly on *Defeat Skeleton* (39.6 percentage points), *Collect Coal* (21.1 percentage points), and *Collect Iron* (10.9 percentage points), consistent with its stronger late-stage progression and higher final normalized environment return. The recurrent comparison shows a broader hierarchical advantage at 20 million steps, particularly for *Make Stone Sword* (72.8 percentage points) and *Make Stone Pickaxe* (60.6 percentage points). As elsewhere, these differences summarise final-update training rates rather than separate greedy evaluation episodes.

B Native Craftax Actions

Table 13: Native action space of Craftax Classic. These actions form the primitive action space in the hierarchical formulation. The DO action executes the context-dependent interaction with the object or entity directly in front of the agent (e.g., mining, attacking, collecting, or drinking).

ID	Action	Description
0	NOOP	Do nothing
1	LEFT	Move left
2	RIGHT	Move right
3	UP	Move up
4	DOWN	Move down
5	DO	Interact with the block or entity in front of the agent
6	SLEEP	Sleep to restore energy
7	PLACE_STONE	Place a stone block
8	PLACE_TABLE	Place a crafting table
9	PLACE_FURNACE	Place a furnace
10	PLACE_PLANT	Plant a sapling
11	MAKE_WOOD_PICKAXE	Craft a wooden pickaxe
12	MAKE_STONE_PICKAXE	Craft a stone pickaxe
13	MAKE_IRON_PICKAXE	Craft an iron pickaxe
14	MAKE_WOOD_SWORD	Craft a wooden sword
15	MAKE_STONE_SWORD	Craft a stone sword
16	MAKE_IRON_SWORD	Craft an iron sword

C Meta-Controller Action Space

C.1 Meta-Action Definitions

Table 14: Full meta-action space of the hierarchical agent. Skill-backed meta-actions invoke a frozen lower-level policy. Direct meta-actions map to native Craftax actions.

ID	Meta-action	Type	Executed action
0	No-op	Direct	`NOOP`
1	Collect Wood	Skill-backed	Collect Wood skill policy
2	Find Water	Skill-backed	Find Water skill policy
3	Hunt Food	Skill-backed	Hunt Food skill policy
4	Combat	Skill-backed	Combat skill policy
5	Collect Stone	Skill-backed	Collect Stone skill policy
6	Collect Coal	Skill-backed	Collect Ore skill policy: coal target
7	Collect Iron	Skill-backed	Collect Ore skill policy: iron target
8	Collect Diamond	Skill-backed	Collect Ore skill policy: diamond target
9	Place table	Direct	`PLACE_TABLE`
10	Place furnace	Direct	`PLACE_FURNACE`
11	Place stone	Direct	`PLACE_STONE`
12	Place plant	Direct	`PLACE_PLANT`
13	Craft wooden pickaxe	Direct	`MAKE_WOOD_PICKAXE`
14	Craft stone pickaxe	Direct	`MAKE_STONE_PICKAXE`
15	Craft iron pickaxe	Direct	`MAKE_IRON_PICKAXE`
16	Craft wooden sword	Direct	`MAKE_WOOD_SWORD`
17	Craft stone sword	Direct	`MAKE_STONE_SWORD`
18	Craft iron sword	Direct	`MAKE_IRON_SWORD`
19	Move up	Direct	`UP`
20	Move left	Direct	`LEFT`
21	Move down	Direct	`DOWN`
22	Move right	Direct	`RIGHT`
23	Sleep	Direct	`SLEEP`

C.2 Formal Meta-Action Set

$$\begin{aligned}\mathcal{M} = \{&\text{No-op}, \text{Collect wood}, \text{Find water}, \text{Hunt food}, \text{Combat},\\ &\text{Collect stone}, \text{Collect coal}, \text{Collect iron}, \text{Collect diamond},\\ &\text{Place table}, \text{Place furnace}, \text{Place stone}, \text{Place plant},\\ &\text{Craft wooden pickaxe}, \text{Craft stone pickaxe}, \text{Craft iron pickaxe},\\ &\text{Craft wooden sword}, \text{Craft stone sword}, \text{Craft iron sword},\\ &\text{Move up}, \text{Move left}, \text{Move down}, \text{Move right}, \text{Sleep}\}. \end{aligned} \tag{9}$$

D Lower-Level Skill Training Specifications

This appendix specifies the objective, bootstrapped environment, observation vector, policy architecture, and OpenES configuration used for each lower-level skill.

D.1 Collect Wood

The Collect Wood skill is trained to gather as much wood as possible within a 20-step rollout. At every reset, the wood inventory is set to zero, all mobs and projectiles are removed, and between one and ten trees are placed within the agent's local field of view. The rollout terminates early if the inventory cap of nine wood is reached.

Let w_t denote the wood inventory after step t and let $T \leq 20$ denote the number of active rollout steps. The fitness is

$$F_{\text{wood}}(\tau) = 10 \sum_{t=0}^{T-1} \max(0, w_{t+1} - w_t)\,. \tag{10}$$

Table 15: Collect Wood skill-specific feature vector. Every feature is derived from the agent's symbolic local field of view; no privileged state is used.

Idx	Feature	Range
0	Normalised row offset to the nearest visible tree	$[-1, 1]$
1	Normalised column offset to the nearest visible tree	$[-1, 1]$
2	Tree visible in the local field of view	$\{0, 1\}$
3	Left neighbouring cell is blocked	$\{0, 1\}$
4	Right neighbouring cell is blocked	$\{0, 1\}$
5	Upper neighbouring cell is blocked	$\{0, 1\}$
6	Lower neighbouring cell is blocked	$\{0, 1\}$
7	Agent is adjacent to a tree	$\{0, 1\}$
8	Agent is facing a tree	$\{0, 1\}$
9	Squared normalised Manhattan distance to the nearest tree	$[0, 1]$
10	Wood inventory normalised by the capacity of nine	$[0, 1]$

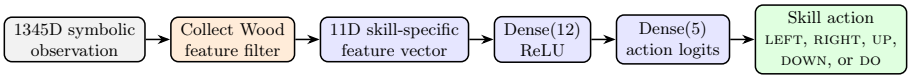


Fig. 21: Collect Wood policy architecture. The feature filter maps the full symbolic observation to an 11-dimensional skill-specific vector, which a 209-parameter multilayer perceptron maps to logits over the five skill actions.

Table 16: OpenES configuration used to train the Collect Wood skill. The final ES mean is selected because the greedy fine-tuning tail matches downstream skill execution.

Configuration	**Value**
Population size	50
Generations	100
Rollout horizon	20 environment steps
Rollouts per candidate	12–27, increased linearly
Rollout-scenario refresh	Every 10 generations
Optimiser	Adam, learning rate 0.05
Search standard deviation	Linear schedule from 0.20 to 0.05
Stochastic action selection	Generations 0–74
Policy temperature	2.0 to approximately 0.88 during the stochastic phase
Greedy fine-tuning tail	Generations 75–99
Training seed	42
Training budget	2,135,000 environment steps
Selected checkpoint	Final ES mean

D.2 Find Water

The Find Water skill is trained to fill the player's drink meter within a 20-step rollout. At reset, all existing water, mobs, and projectiles are removed, the drink and thirst variables are set to zero, and one water tile is placed at a Chebyshev distance of two to four cells from the player. The rollout terminates when the drink meter reaches its capacity of nine.

Let d_t denote the drink level after step t and let $T \leq 20$ denote the number of active rollout steps. The fitness is

$$F_{\text{water}}(\tau) = 10 \sum_{t=0}^{T-1} \max(0, d_{t+1} - d_t)\,. \tag{11}$$

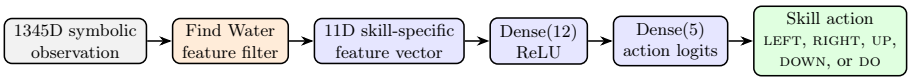


Fig. 22: Find Water policy architecture. The 11-dimensional skill-specific vector is processed by a 209-parameter multilayer perceptron.

Table 17: Find Water skill-specific feature vector. Every feature is derived from the agent's symbolic local field of view.

Idx	Feature	Range
0	Normalised row offset to the nearest visible water tile	$[-1, 1]$
1	Normalised column offset to the nearest visible water tile	$[-1, 1]$
2	Water visible in the local field of view	$\{0, 1\}$
3	Left neighbouring cell is blocked	$\{0, 1\}$
4	Right neighbouring cell is blocked	$\{0, 1\}$
5	Upper neighbouring cell is blocked	$\{0, 1\}$
6	Lower neighbouring cell is blocked	$\{0, 1\}$
7	Agent is adjacent to water	$\{0, 1\}$
8	Agent is facing water	$\{0, 1\}$
9	Squared normalised Manhattan distance to the nearest water tile	$[0, 1]$
10	Drink level normalised by the capacity of nine	$[0, 1]$

Table 18: OpenES configuration used to train the Find Water skill. The final ES mean is selected because the greedy fine-tuning tail matches downstream skill execution.

Configuration	Value
Population size	100
Generations	100
Rollout horizon	20 environment steps
Rollouts per candidate	8–50, increased linearly
Rollout-scenario refresh	Every 10 generations
Optimiser	Adam, learning rate 0.05
Search standard deviation	Linear schedule from 0.20 to 0.05
Stochastic action selection	Generations 0–74
Policy temperature	2.0 to approximately 0.88 during the stochastic phase
Greedy fine-tuning tail	Generations 75–99
Training seed	42
Training budget	6,840,000 environment steps
Selected checkpoint	Final ES mean

D.3 Hunt Food

The Hunt Food skill is trained to pursue and kill cows within a 90-step rollout. Each bootstrapped map consists of grass with between one and three cows and zero to nine trees placed within the local field of view; hostile mobs and projectiles are removed and the player receives a wooden sword. Training uses two 100-generation phases. Cows are stationary in the first phase and move normally in the second, which is warm-started from the final ES mean of the first phase.

Let k_t be the number of cows killed at step t, h_t indicate that `DO` damages a cow, u_t indicate an unsuccessful `DO` action, and a_t indicate that at least one cow remains alive. Furthermore, let $\phi_t = 1/(d_t + 1)$, where d_t is the distance to the nearest cow, and let $\Delta\phi_t = \phi_{t+1} - \phi_t$, with this term set to zero on kill steps or when no cow exists before or after the transition. For $T \leq 90$ active steps, the fitness is

$$F_{\text{food}}(\tau) = \sum_{t=0}^{T-1} \left(10k_t + 0.5h_t + 5\Delta\phi_t - 0.02u_t - 0.05a_t\right). \tag{12}$$

Table 19: Hunt Food skill-specific feature vector. Every feature is derived from the agent's symbolic local field of view.

Idx	Feature	Range
0	Normalised row offset to the nearest visible cow	$[-1, 1]$
1	Normalised column offset to the nearest visible cow	$[-1, 1]$
2	Cow visible in the local field of view	$\{0, 1\}$
3	Left neighbouring cell is blocked	$\{0, 1\}$
4	Right neighbouring cell is blocked	$\{0, 1\}$
5	Upper neighbouring cell is blocked	$\{0, 1\}$
6	Lower neighbouring cell is blocked	$\{0, 1\}$
7	Agent is adjacent to a cow	$\{0, 1\}$
8	Agent is facing a cow	$\{0, 1\}$
9	Squared normalised Manhattan distance to the nearest cow	$[0, 1]$

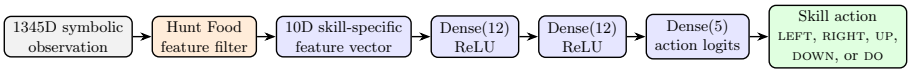


Fig. 23: Hunt Food policy architecture. The 10-dimensional skill-specific vector is processed by a 353-parameter multilayer perceptron.

Table 20: OpenES configuration used to train the Hunt Food skill. The second phase is warm-started from the first, and its final ES mean is used downstream.

Configuration	**Value**
Training phases	100 generations with stationary cows, followed by 100 generations with moving cows
Population size	100 per phase
Rollout horizon	90 environment steps
Rollouts per candidate	8–105 per phase, increased linearly
Rollout-scenario refresh	Every 12 generations
Optimiser	Adam, learning rate 0.05
Search standard deviation	Linear schedule from 0.20 to 0.05 in each phase
Stochastic action selection	Generations 0–79 of each phase
Policy temperature	2.0 to approximately 0.80 during each stochastic phase
Greedy fine-tuning tail	Generations 80–99 of each phase
Training seed	42
Training budget	118,260,000 environment steps
Selected checkpoint	Final ES mean of the moving-cow phase

D.4 Combat

The Combat skill is trained to kill zombies and skeletons within a 90-step rollout. Each bootstrapped map consists of grass with one to three zombies, zero to two skeletons, and zero to nine trees placed within the local field of view; cows and existing projectiles are removed and the player receives a wooden sword. Training uses two 100-generation phases. Zombies are stationary in the first phase and move normally in the second, which is warm-started from the final ES mean of the first phase; skeletons retain their normal ranged behaviour in both phases.

Let k_t be the number of hostiles killed at step t, h_t indicate that `DO` damages a hostile, u_t indicate an unsuccessful `DO` action, and a_t indicate that at least one hostile remains alive. Let $\phi_t = 1/(d_t + 1)$ for distance d_t to the nearest hostile and $\Delta\phi_t = \phi_{t+1} - \phi_t$, with this term set to zero on kill steps or when no hostile exists before or after the transition. For $T \leq 90$ active steps, the fitness is

$$F_{\text{combat}}(\tau) = \sum_{t=0}^{T-1} \left(10k_t + 0.5h_t + 5\Delta\phi_t - 0.02u_t - 0.05a_t\right). \tag{13}$$

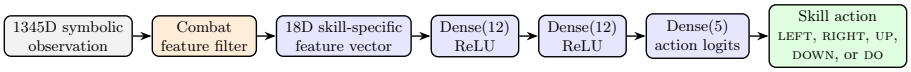


Fig. 24: Combat policy architecture. The 18-dimensional skill-specific vector is processed by a 449-parameter multilayer perceptron.

Table 21: Combat skill-specific feature vector. Hostile and projectile features are restricted to the agent's symbolic local field of view.

Idx	Feature	Range
0	Normalised row offset to the nearest visible hostile	$[-1, 1]$
1	Normalised column offset to the nearest visible hostile	$[-1, 1]$
2	Inverse Manhattan distance $1/(d+1)$ to that hostile	$[0, 1]$
3	Hostile visible in the local field of view	$\{0, 1\}$
4	Left neighbouring cell is blocked	$\{0, 1\}$
5	Right neighbouring cell is blocked	$\{0, 1\}$
6	Upper neighbouring cell is blocked	$\{0, 1\}$
7	Lower neighbouring cell is blocked	$\{0, 1\}$
8	Agent is adjacent to a hostile	$\{0, 1\}$
9	Agent is facing a hostile	$\{0, 1\}$
10	Player health normalised by the maximum of nine	$[0, 1]$
11	Nearest hostile is a skeleton	$\{0, 1\}$
12	Arrow visible in the local field of view	$\{0, 1\}$
13	Normalised row offset to the nearest visible arrow	$[-1, 1]$
14	Normalised column offset to the nearest visible arrow	$[-1, 1]$
15	Inverse Manhattan distance $1/(d+1)$ to that arrow	$[0, 1]$
16	Visible-arrow count, capped and normalised by three	$[0, 1]$
17	Arrow occupies the cell faced by the player	$\{0, 1\}$

Table 22: OpenES configuration used to train the Combat skill. The second phase is warm-started from the first, and its final ES mean is used downstream.

Configuration	Value
Training phases	100 generations with stationary zombies, followed by 100 generations with moving zombies
Population size	100 per phase
Rollout horizon	90 environment steps
Rollouts per candidate	8–105 per phase, increased linearly
Rollout-scenario refresh	Every 12 generations
Optimiser	Adam, learning rate 0.05
Search standard deviation	Linear schedule from 0.20 to 0.05 in each phase
Stochastic action selection	Generations 0–79 of each phase
Policy temperature	2.0 to approximately 0.80 during each stochastic phase
Greedy fine-tuning tail	Generations 80–99 of each phase
Training seed	42
Training budget	118,260,000 environment steps
Selected checkpoint	Final ES mean of the moving-zombie phase

D.5 Collect Stone

The Collect Stone skill is trained to gather stone within a 20-step rollout. At reset, mobs and projectiles are removed, naturally generated ore blocks are replaced by grass, and between two and twelve stone blocks are placed within the local field of view. The player receives a wooden pickaxe, the stone inventory is set to zero, and the rollout terminates when the inventory reaches its capacity of nine.

Let s_t denote the stone inventory after step t and let $T \leq 20$ denote the number of active rollout steps. The fitness is

$$F_{\text{stone}}(\tau) = 10 \sum_{t=0}^{T-1} \max(0, s_{t+1} - s_t)\,. \tag{14}$$

Table 23: Collect Stone skill-specific feature vector. Every feature is derived from the agent's symbolic local field of view.

Idx	Feature	Range
0	Normalised row offset to the nearest visible stone block	$[-1, 1]$
1	Normalised column offset to the nearest visible stone block	$[-1, 1]$
2	Stone visible in the local field of view	$\{0, 1\}$
3	Left neighbouring cell is blocked	$\{0, 1\}$
4	Right neighbouring cell is blocked	$\{0, 1\}$
5	Upper neighbouring cell is blocked	$\{0, 1\}$
6	Lower neighbouring cell is blocked	$\{0, 1\}$
7	Agent is adjacent to stone	$\{0, 1\}$
8	Agent is facing stone	$\{0, 1\}$
9	Squared normalised Manhattan distance to the nearest stone block	$[0, 1]$
10	Stone inventory normalised by the capacity of nine	$[0, 1]$

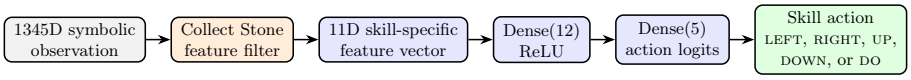


Fig. 25: Collect Stone policy architecture. The 11-dimensional skill-specific vector is processed by a 209-parameter multilayer perceptron.

Table 24: OpenES configuration used to train the Collect Stone skill. The final ES mean is selected because the greedy fine-tuning tail matches downstream skill execution.

Configuration	**Value**
Population size	100
Generations	100
Rollout horizon	20 environment steps
Rollouts per candidate	8–50, increased linearly
Rollout-scenario refresh	Every 12 generations
Optimiser	Adam, learning rate 0.05
Search standard deviation	Linear schedule from 0.20 to 0.05
Stochastic action selection	Generations 0–74
Policy temperature	2.0 to approximately 0.88 during the stochastic phase
Greedy fine-tuning tail	Generations 75–99
Training seed	42
Training budget	6,840,000 environment steps
Selected checkpoint	Final ES mean

D.6 Collect Ore

The Collect Ore skill is trained on coal and reused for iron and diamond by changing the target block and inventory channels extracted from the symbolic observation. During training, mobs, projectiles, naturally generated ores, and ambient stone are removed. A coal block is placed three or four cells horizontally from the player and enclosed by one to three layers of stone. The player receives wooden and stone pickaxes and begins with empty ore inventories. The wall curriculum initially uses one layer and progressively increases the probability of two- and three-layer layouts after 40% and 70% of training, respectively.

Let q_t denote the target-ore inventory after step t and let b_t indicate that a stone tile is broken at step t. For the ten-step rollout, the fitness is

$$F_{\text{ore}}(\tau) = \sum_{t=0}^{T-1} \left[10 \max(0, q_{t+1} - q_t) + 0.5 b_t\right], \qquad T \leq 10. \tag{15}$$

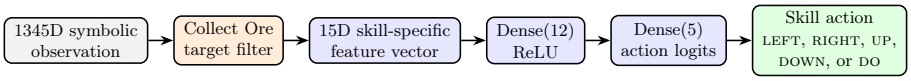


Fig. 26: Collect Ore policy architecture. The target-conditioned 15-dimensional vector is processed by a 257-parameter multilayer perceptron shared across coal, iron, and diamond.

Table 25: Collect Ore skill-specific feature vector. At deployment, the same layout is instantiated for coal, iron, or diamond.

Idx	Feature	Range
0	Normalised row offset to the nearest visible target ore	$[-1, 1]$
1	Normalised column offset to the nearest visible target ore	$[-1, 1]$
2	Target ore visible in the local field of view	$\{0, 1\}$
3	Left neighbouring cell is blocked	$\{0, 1\}$
4	Right neighbouring cell is blocked	$\{0, 1\}$
5	Upper neighbouring cell is blocked	$\{0, 1\}$
6	Lower neighbouring cell is blocked	$\{0, 1\}$
7	Agent is adjacent to the target ore	$\{0, 1\}$
8	Agent is facing the target ore	$\{0, 1\}$
9	Squared normalised Manhattan distance to the target ore	$[0, 1]$
10	Target inventory normalised by its collection cap	$[0, 1]$
11	Normalised row offset to the nearest intervening stone block	$[-1, 1]$
12	Normalised column offset to the nearest intervening stone block	$[-1, 1]$
13	Agent is adjacent to stone	$\{0, 1\}$
14	Agent is facing stone	$\{0, 1\}$

Table 26: OpenES configuration used to train the Collect Ore skill. The final coal-trained ES mean is reused for all three deployment targets.

Configuration	Value
Training target	Coal
Deployment targets	Coal, iron, and diamond
Population size	100
Generations	200
Rollout horizon	10 environment steps
Rollouts per candidate	8–50, increased linearly
Rollout-scenario refresh	Every 12 generations
Wall curriculum	Increasing probability of one-, two-, and three-layer stone shells
Optimiser	Adam, learning rate 0.05
Search standard deviation	Linear schedule from 0.20 to 0.05
Stochastic action selection	Generations 0–149
Policy temperature	2.0 to approximately 0.88 during the stochastic phase
Greedy fine-tuning tail	Generations 150–199
Training seed	42
Training budget	6,846,000 environment steps
Selected checkpoint	Final ES mean

E Craftax Classic Observation Structure

E.1 Observation-Vector Overview

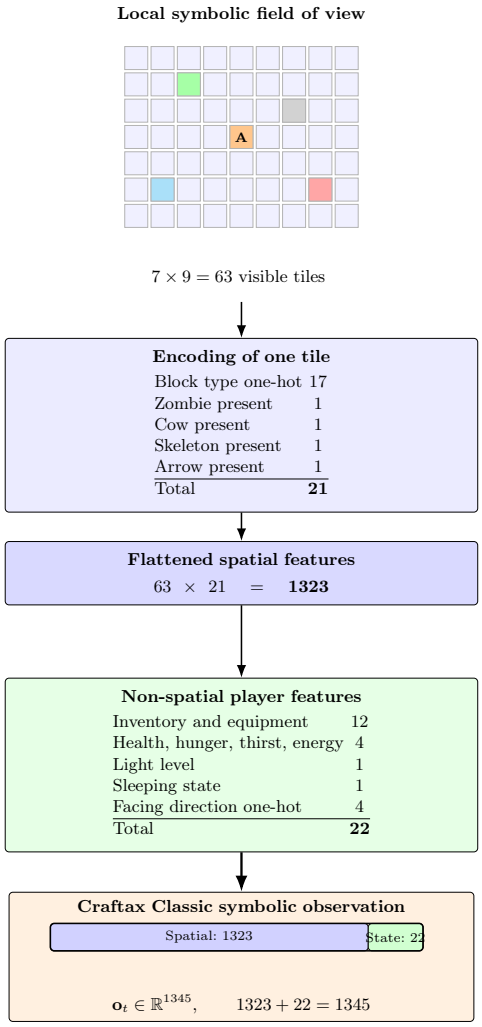


Fig. 27: Structure of the 1345-dimensional symbolic observation used in Craftax Classic. A local 7×9 field of view is encoded tile-wise and flattened into 1323 spatial features, which are concatenated with 22 non-spatial player-state features.

E.2 Spatial-Layer Overview

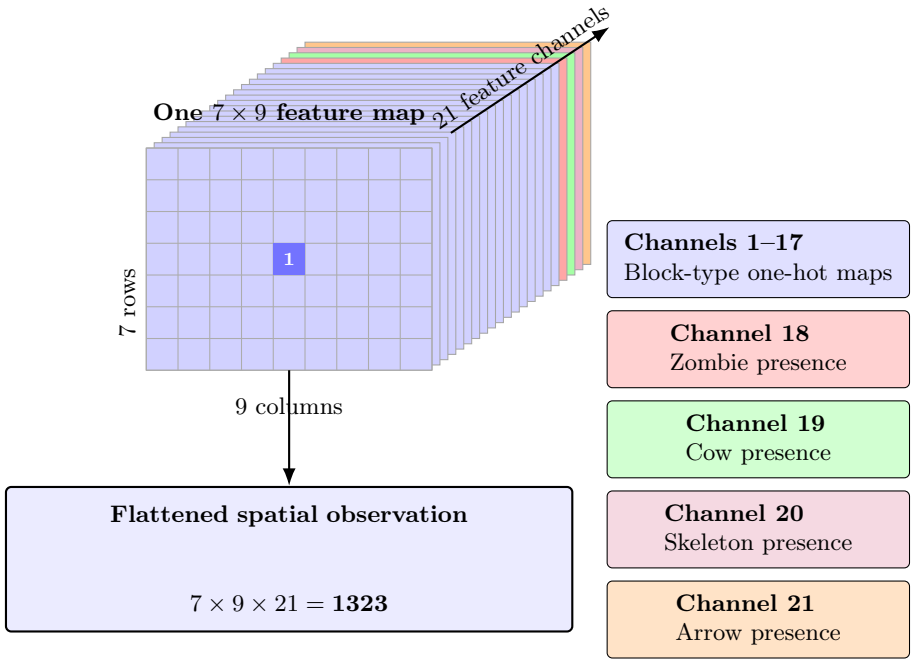


Fig. 28: Channel-wise representation of the spatial component of the Craftax Classic symbolic observation. Each of the 21 channels is a 7×9 feature map. The first 17 channels encode block types, while the remaining four indicate the presence of zombies, cows, skeletons, and arrows. Flattening the tensor produces 1323 spatial features.

E.3 Block-Type Channels

Table 27: Block-type channels in the Craftax Classic symbolic observation.

Index	Block type
0	INVALID
1	OUT_OF_BOUNDS
2	GRASS
3	WATER
4	STONE
5	TREE
6	WOOD
7	PATH
8	COAL
9	IRON
10	DIAMOND
11	CRAFTING_TABLE
12	FURNACE
13	SAND
14	LAVA
15	PLANT
16	RIPE_PLANT

F Skill-Specific Feature-Vector Pipeline

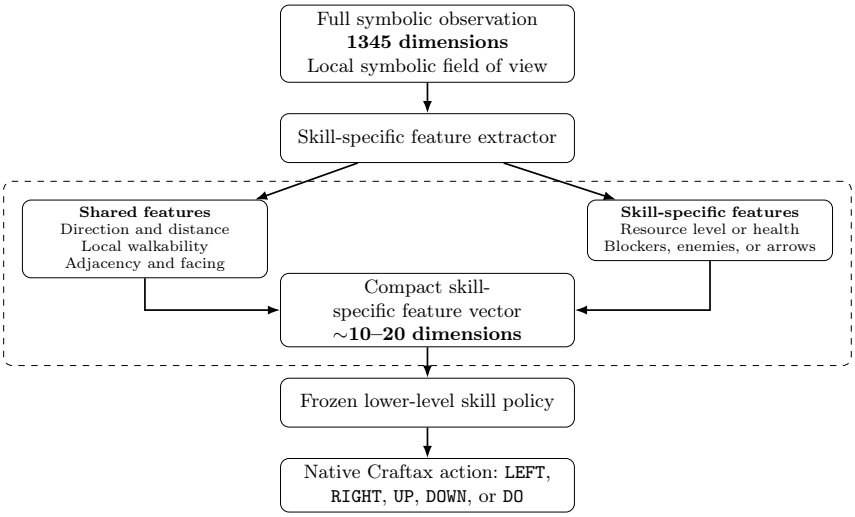


Fig. 29: Construction of the skill-specific feature vector used by each lower-level skill. A skill-specific feature extractor compresses the full symbolic observation into a compact vector containing shared geometric features and skill-specific state information. All features are derived exclusively from the agent's local symbolic field of view.

G Experimental Hardware and Software Environment

All experiments were run on a single ASUS ROG Strix GL504GV laptop. The configuration in Table 28 contextualises the reported wall-clock measurements.

Table 28: Hardware and software environment used for the experiments.

Component	Specification
Laptop model	ASUS ROG Strix GL504GV
CPU	Intel Core i7-8750H at 2.20 GHz (6 cores, 12 threads)
Memory	16 GB DDR4-2667
Discrete GPU	NVIDIA GeForce RTX 2060 (6 GB GDDR6)
Host OS	Microsoft Windows 11 Home (64-bit, build 26200)
Virtualisation	WSL2
WSL2 kernel	`5.15.167.4-microsoft-standard-WSL2`
Guest OS	Ubuntu 24.04.4 LTS
GPU software	NVIDIA driver 555.97; CUDA 12.5